HOME MINISTRY

(Part-I)



Developed By

Saima Nawaz

1594-FBAS/BSSE/F11

MamoonaSafdar

1573-FBAS/BSSE/F11

BS Software Engineering

Supervisor

Ms. Sadia Arshad

Department of Computer Science & Software Engineering

Faculty of Basic and Applied Sciences

International Islamic University, Islamabad

(2015)

Dedication

This project is dedicated to our family our teachers our friends and all well-wishers who give us courage and strength to be at this point.

Ms. Saima Nawaz

(1594-FBAS/BSSE/F11B)

Ms. Mamoona Safdar

(1573-FBAS/BSSE/F11B)

Declaration

We hereby declare that this Software, neither as a whole nor as a part thereof has been copied out from any source. It is further declared that we have developed this software for real client under the sincere guidance of our supervisor and co-supervisor. No portion of the work presented in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.

Ms. Saima Nawaz

(1594-FBAS/BSSE/F11B)

Ms. Mamoona Safdar

(1573-FBAS/BSSE/F11B)

Acknowledgement

All praise to Almighty Allah, who gave us the understanding, courage and patience to complete this project.

We want to thank all those people who helped us in our project. Without their contribution we will never be able to complete it on time. Our families also encouraged us a lot in completing the project. No doubt without all these people we were not able to complete our task in a short time.

We express our gratitude to our kind teacher Ms. Sadia Arshad for providing us the opportunity to learn and enhance our knowledge. She had been ready to help and guide us throughout the project.

Special Thanks to Mr Bilal and Mr Abdul Rehman who co-supervised the project for helping us in understanding of project and their proper guidance in implementation. They were very supportive during whole project.

Ms. Saima Nawaz

(1594-FBAS/BSSE/F11B)

Ms. Mamoona Safdar

(1573-FBAS/BSSE/F11B)

Project in Brief

Project Title	Home Ministry
Objective	The application is developed to assist the house wife and working lady in management of home tasks.
Undertaken By:	Mamoon asafdar Saima Nawaz
Supervised By:	Ms. Sadia Arshad
Date Started:	Fall 2014
Date Completed:	Spring 2015
Tools Used:	Windows 8 OS, Android studio, SQL lite ,pencil, GIMP 2

Abstract

Home Ministry is an android application, which provides managing facility to user who manages home tasks. Through this application a user is capable of managing budget, laundry and deciding dish of the day. A user does not have to remember expenses, income or laundry detail. If this application is not installed in a user's phone, then user have to manually keep track or remember things. This application help to manage budget, keep record of income and expenses. And show their total. Graphical display is also shown in form of pie charts. This application help to manage laundry items sent for washing, dry cleaning and pressing. And also help to create menu of the week and send message to relatives to decide dish of the day. This application is developed basically by using android development tool and concepts.

Table of Contents

1 Introduction 2
1.1 Project Motivation 2
1.2 Agreement: 2
1.3 Overview 3
1.4 Purpose 3
1.5 Objectives 3
1.6 Problem Statement 4
1.7 Product Problem Statement: 5
1.8 Stakeholder and User Description 5
1.9 User Summary 5
1.10 Alternatives and Competition 6
2 Problem analysis 22
2.1 Project Idea 22
2.2 Proposed System 22
2.3 Inception Phase 22
2.4 Elaboration Phase 22
2.5 Construction Phase 23
2.6 Transition Phase 23
2.7 Software Technology 23
2.8 Use Case Diagram 24
2.9 Fully Dressed Use Cases 26
2.10 Sign Up 26
2.11 Log in 28
2.12 Add income 30
COMMENTS 30
2.13 View income 31
2.14 Edit Income 32
2.15 Delete Income 34
2.16 View summary 35

2.17 Add Expenses....36
2.18 Edit Expenses....38
2.19 View Expenses....39
2.20 Delete Expenses....40
2.21 View Reports....42
2.22 View budgeting....43
2.23 View savings....44
2.24 Add laundry....45
2.25 Edit laundry....47
2.26 Delete Laundry....48
2.27 View Laundry....50
2.28 Set Price....51
2.29 Add Relatives....52
2.30 Create menu....54
2.31 Send messages....55
2.32 System Sequence Diagrams....57
2.32.1 Add income....57
2.32.2 Edit income....59
2.32.3 Delete income....60
2.32.4 View income....61
2.32.5 Add expenses....62
2.32.6 Edit expenses....62
2.32.7 Delete expenses....63
2.32.8 Add laundry....65
2.32.9 Edit laundry....66
2.32.10 Delete laundry....67
2.32.11 View summary....69
2.32.12 View savings....70
2.32.13 View budget....71
2.32.14 View reports....72
2.32.15 Add relative....73
2.32.16 Send messages....74

2.32.17 Set menu 75
2.33 Domain Model 76
3 System Analysis 79
3.1 Specific Functional Requirement 79
3.1.1 User Authentication 79
3.1.2 Budgeting 79
3.1.3 Laundry 80
3.1.4 Meal today 80
3.2 Nonfunctional Requirements 80
3.2.1 Performance Requirements 80
3.2.2 Portability Machine 81
3.2.3 Language 81
3.2.4 Availability 81
3.2.5 Continuity 81
3.2.6 Reliability 81
3.2.7 Usability 81
3.3 User Characteristics 81
4 System Design 83
4.1 Class Diagram 83
5 Implementation: 85
5.1 Software Development Lifecycle Model: 85
5.2 Software Development Tool: 85
5.3 Pseudo codes: 85
5.3.1 Pseudo code Login 86
5.3.2 Pseudo code Budgeting 86
5.3.3 Pseudo code laundry 87
5.3.4 Pseudo code set menu 88
5.3.5 Pseudo code Add relatives 89
5.3.6 Pseudo code send messages 89
5.3.7 Set prices 89
5.3.8 View savings 90
5.3.9 View reports 90

Select view reports 90
5.3.10 View summary 90
5.4 Deployment Phase 91
6 Testing 96
6.1 Test Cases 96
6.1.1 Login 96
6.1.2 Login 97
6.1.3 Sign up 98
6.1.4 Sign up 98
6.1.5 Add income 100
6.1.6 View income 101
6.1.7 Delete income 101
6.1.8 Add expenses 102
6.1.9 Add expenses 103
6.1.10 View expenses 104
6.1.11 Delete expenses 105
6.1.12 Add laundry 106
6.1.13 Delete laundry 107
6.1.14 View laundry 108
6.1.15 Add relatives 109
6.1.16 Set menu 110
6.1.17 View summary 111
6.1.18 Send messages 112
7 Conclusion 115
APPENDIX A 116
APPENDIX B 127
1. Diagramming and Collaboration, August, 2015. URL:www.creately.com 128

CHAPTER 1

INTRODUCTION

1 Introduction

There is an abundance of management systems being developed as final year projects and occasionally games also but there is hardly ever a system developed that contributes to the household. There are office management systems and business management systems but there is not any management system that helps the housewives. Who still did not have any system that can help them for their daily household tasks.

Almost every system developed as a Final Year Project offers either some or no actual benefit to the public or they are dull and boring so they don't attract much audience. So we have developed android application which will help ease the burden and responsibilities of a housewife i.e. the application can actually be used to make a difference in everyday lives of a housewife. The project is an android application which contains the features that a housewife majorly has to deal with i.e. budgeting, laundry and menu management.

1.1 Project Motivation

The inspiration and incentive for this application came from the existing application for budgeting management but none of them are implementing household tasks at once place in an android application. And none of them were made for the purpose or intent to give some relieves the housewives of their burdens or duties.

In fact no system has ever been made for this intent so this is an innovative application which can do some assistance in many works a housewife does every day.

1.2 Agreement:

This application is a Final Year Project for the University and it will be considered as the **POC** by the Beacon Light Software Solutions.

1.3 Overview

Home ministry consists of management of basic house hold tasks which is difficult and time consuming. This application assist user to done these task efficiently with minimum time and more accurately as compare to manually done by user. The application contain three basic modules which consist of budgeting, laundry, and dish of the day .In budgeting user easily manage their income savings and expenses. In laundry user easily manage their receiving date, prices and number of items. In cooking user easily set their menu and decide dish of the day according to the choice of their family members.

1.4 Purpose

The purpose of the Home Ministry is that user can deal with their house hold task more precisely.

None of any application who provide all functionality of home management in one plate form Home Ministry provides all functionality like budgeting, laundry, cooking. User manage their basic task through one application like manage their monthly budget, laundry, cooking which are somehow difficult for any house hold user.

1.5 Objectives

Project aims and objectives that are achieved after the completion of the project were carried out in this sub chapter. The succession of the application is also evaluated through this sub chapter.

Following are objectives of this project.

- To provide functionality to manage house hold tasks.
- To provide full functionality to calculate budget with accuracy and in less time.
- To provide functionality to manage laundry properly.
- To assist the user to decide Meal today which is a difficult task for a house hold user.
- To provides functionality to maintain expenses that should not be exceed from income.
- To provides functionality to calculate savings.
- To provide graphical view of income and expenses

1.6 Problem Statement

Problem statement is given below:

The problem of	Loss of proper management of budgeting, laundry, meal.
Affects	Calculation against all home management is quite difficult and time consuming. And keeping track for laundry is difficult. And deciding dish of the day was difficult.
the impact of which is	If management is not done properly it would effect.
a successful solution would be	The application will give proper management against different task of House wife like budgeting, laundry and related to meal. This application properly assists the House Wife in management task properly.

Table 1.1: Problem statement

1.7 Product Problem Statement:

Problem statement is given below:

For	Any user that have android phone
Home Ministry	Is a Home management related application
That	Helps user manage their task and maintain data.
Unlike	Existing android management phone applications.
Our product	Will facilitate user to manage their difficult task of management like budgeting, laundry and meal today.

Table 1.2 Product Problem Statement of Home Ministry

1.8 Stakeholder and User Description

This project is basically for users who install this application in their android phone. It helps them to manage home task.

1.9 User Summary

Name	Description	Stakeholder
House wife	End user	Owner

Table 1.2: User Summary

1.10 Alternatives and Competition

Some alternatives are available, but they don't have a collection of all management system for home task in one application. The main difference is that this application contains budgeting, laundry, meal today management in one platform which has considerable attention. This application provide management of budget precisely on basis of income and expenses, then laundry management, and dish of the decision is made through this application. None of the application provides this type of functionality.

CHAPTER 2

PROBLEM ANALYSIS

2 Problem analysis

Problem analysis is the process of understanding real world problems, user's needs and proposing solutions to meet those needs. The goal of problem analysis is to gain better understanding of a problem being solved, before development begins.

2.1 Project Idea

Project idea is to develop an android application which can facilitate any house hold lady who wants to manage her home task appropriately with in less time and with accuracy.

2.2 Proposed System

The proposed idea of Home Ministry is to develop an android application to help out the user for managing their daily basis house hold tasks. The main idea to calculate income and expenses for current month .It also help out the user to show income expenses and saving graphically. User can manage laundry. It also facilitates the user to create menu of the weak and decide dish for the day through messages.

2.3 Inception Phase

During Inception phase problem is identified and analyzed and a solution is proposed for the solving the problem in the form of a proposed system. In Inception phase, the problem is identified and solution is proposed for it.

2.4 Elaboration Phase

Elaboration phase is where the project starts to take shape. In this phase the problem domain analysis is done and the architecture of the project gets its basic form. The actors of the project

were modeled and their goals recognized in inception phase in a use-case model to identify the use cases of "Home Ministry". Use case descriptions are also developed during this phase. A description of the software architecture was finalized. A development plan was made for the whole project. System design has been made in this phase. For System design UML diagrams System Sequence Diagrams were made.

2.5 Construction Phase

System has been implemented in this phase. Testing is also done in this phase. For testing purpose test cases were developed.

2.6 Transition Phase

Transition phase involves the deployment and product services of the system developed in construction phase.

2.7 Software Technology

This Application was developed by using following tools

- Windows Operating System
- Android studio
- SQL LITE
- Gimp2, basic Photoshop.
- Pencil
- Microsoft Office
- Visual paradigm (tool for UML diagram).

2.8 Use Case Diagram

The use case diagram of project is shown in figure 2.1

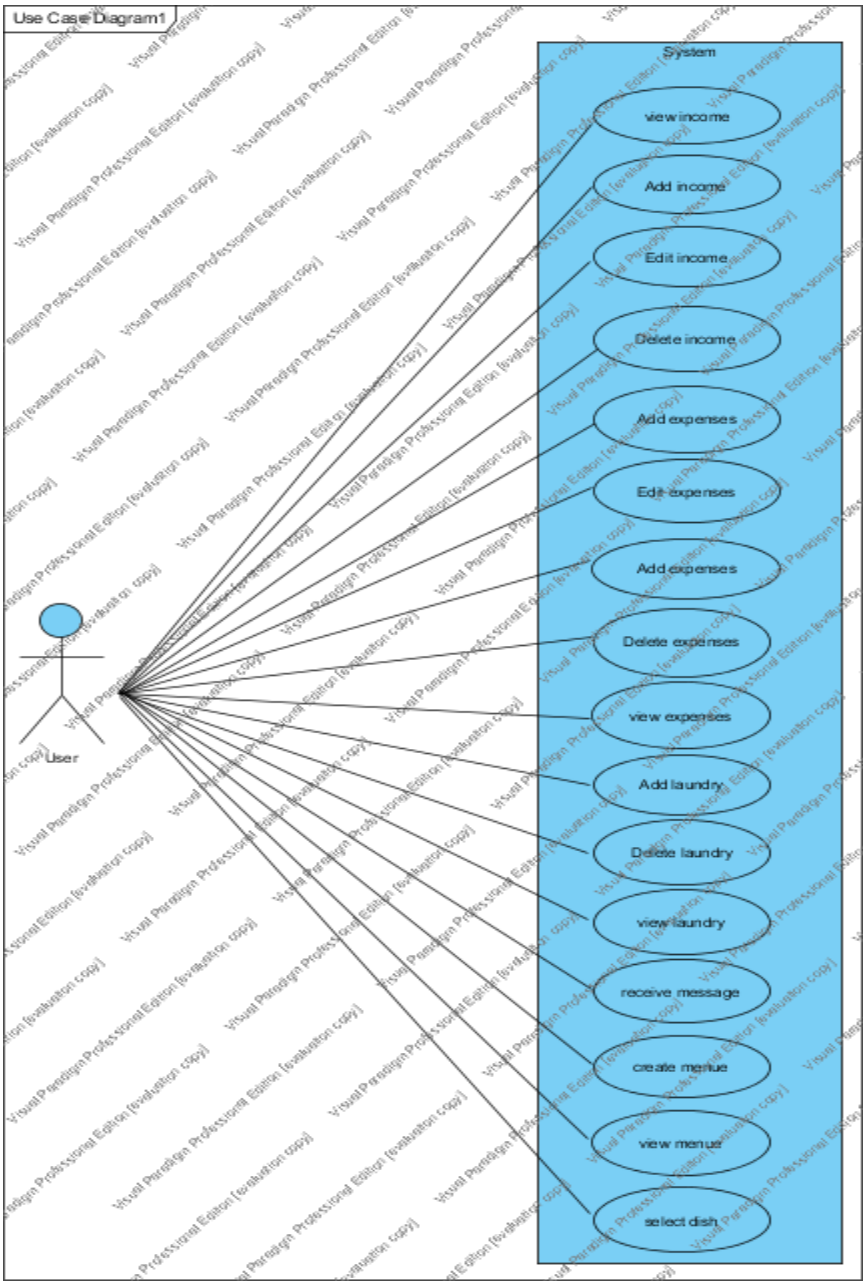


Figure 2.1: Use case diagram of The Home Ministry

2.9 Fully Dressed Use Cases

Fully dressed use cases of this project are as follow

2.10 Sign Up

UC-001	
USE CASE SECTION	**COMMENTS**
Use case Name	Register a user
Scope	Everyone
Level	The user can track house hold management task.
Primary Actor	User
Preconditions	Application should be installed
Success Guarantee	User should be added successfully
Main Success Scenario	User Action System Action
	1.Click on the “sign up button” 2. User enters Information. 3. User click “Register” button.

		4.Application saves the information
Extensions	User Action	System Action
	1. Click on the “register users button” 2. User enters the required information. 3. Userclick “Register” button.	4.Appliaction gives error message “incorrect information” 5. Application redirect to the same page.
Special Requirements	There is no special requirement for this use case	

Frequency Of Occurrence	Very high

Table 2.1: Use Case for Sign up

2.11 Log in

UC-002		
USE CASE SECTION	**COMMENTS**	
Use case Name	Log in	
Scope	Everyone	
Level	The user can track house hold management task.	
Primary Actor	User	
Preconditions	Application should be installed	
Success Guarantee	User should be added successfully	
Main Success Scenario	User Action	System Action
	1.User select to open the application	2.System lounge application and prompts to enter user name and password
	3.User enters name And password	
	4. User click “Log in”	

	button.	5.System logged the user.
Extensions	User Action	System Action
	1. User enters name 2. User enters password. 3. User click “Log in” button.	4.Appliaction gives error message “incorrect password” 5. Application redirect to the same page.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Very high	

Table 2.2: Use Case for Log in

2.12 Add income

UC-003		
USE CASE SECTION	**COMMENTS**	
Use case Name	Add Income	
Scope	Everyone	
Level	So that user can add income.	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	Income will be added in a system	
Main Success Scenario	User Action	System Action
	1. User will select income from main menu.	
		2. System displays the income window.
	3. User selects to add income.	
		4. System prompts to fill required fields.
	5. User will enter the required data.	
		6. System will verify the data and will save

		income added.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.3: Use Case for Add income

2.13 View income

UC-004	
USE CASE SECTION	**COMMENTS**
Use case Name	View Income
Scope	Everyone
Level	So that user can View income.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Income will be View in a system
Main Success Scenario	User Action System Action

	1. User will select income from main menu. 3. User selects already added income and will select to View.	2. System displays the income window. 4. System displays all details of Income.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.4: Use Case View Income

2.14 Edit Income

UC-005	
USE CASE SECTION	**COMMENTS**
Use case Name	Edit Income
Scope	Everyone
Level	So that user can add income.
Primary Actor	User
Preconditions	Application should be installed.

Success Guarantee	Income will be edited in a system	
Main Success Scenario	User Action	System Action
	1. User will select income from main menu. 3. User selects already added income and will select to edit. 5. User will enter the required data.	2. System displays the income window. 4. System prompts to fill required fields. 6. System will verify the data and will save income edited.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.5: Use Case to Edit Income

2.15 Delete Income

UC-006	
USE CASE SECTION	**COMMENTS**
Use case Name	Delete Income
Scope	Everyone
Level	So that user can delete income.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Income will be deleted from a system
Main Success Scenario	User Action System Action
	1. User will select income from main menu. 2. System displays the income window. 3. User selects to income to be deleted. 4. System will remove the selected income and will notify to user.
Special Requirements	There is no special requirement for this use

	case
Frequency Of Occurrence	Medium

Table 2.6: Use Case to Delete Income

2.16 View summary

UC-007	
USE CASE SECTION	**COMMENTS**
Use case Name	View summary
Scope	Everyone
Level	So that user can view summary.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Summary is displayed by the system
Main Success Scenario	User Action System Action
	1. User will select summary from main menu. 2. System displays the window and prompts to enter month and year. 3. User adds the asked

	data.	4. System displays summary of income and expenses.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.7: Use Case view summary

2.17 Add Expenses

UC-008	
USE CASE SECTION	**COMMENTS**
Use case Name	Add expenses
Scope	Everyone
Level	So that user can add expenses
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Expenses will be added in a system

Main Success Scenario	User Action	System Action
	1. User will select expenses from main menu. 3. User will select the category. 5. User will enter the required data.	2. System displays the expenses window and will prompt to select category. 4. System prompts to fill required fields. 6. System will verify the data and will save expenses added.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.8: Use Case Add expenses

2.18 Edit Expenses

UC-009	
USE CASE SECTION	**COMMENTS**
Use case Name	Edit Expenses
Scope	Everyone
Level	So that user can edit expenses.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Expenseswill be edited in the system
Main Success Scenario	User Action / System Action
	1. User will select expenses from main menu. 2. System displays the expense window. 3. User selects the already present expenses. 4. System will display detail of expense selected. 5. User will select to edit expense. 6. System will

	7. User will edit data.	display editing form. 8. System will verify data and will save it.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.9: Use Case Edit Expenses

2.19 View Expenses

UC-010	
USE CASE SECTION	**COMMENTS**
Use case Name	View expenses
Scope	Everyone
Level	So that user can View expenses.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Expenses will be View in a system
Main Success Scenario	User Action System Action

	1. User will select expenses from main menu. 3. User selects already added expenses and will select to View.	2. System displays the expenses window. 4. System displays all details of expenses.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.10: Use Case View Expenses

2.20 Delete Expenses

UC-011

USE CASE SECTION	**COMMENTS**	
Use case Name	Delete Expenses	
Scope	Everyone	
Level	So that user can delete expenses.	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	Expenses will be deleted in the system	
Main Success Scenario	User Action	System Action
	1. User will select expenses from main menu. 3. User selects the already present expenses. 5. User will select to delete expense.	2. System displays the expense window. 4. System will display detail of expense selected. 6. System will delete the selected expense.
Special Requirements	There is no special requirement for this use case	

Frequency Of Occurrence	Medium

Table 2.10: Use Case Delete Expenses

2.21 View Reports

UC-011	
USE CASE SECTION	**COMMENTS**
Use case Name	View Reports
Scope	Everyone
Level	So that user can view summary.
Primary Actor	user
Preconditions	Application should be installed.
Success Guarantee	Summary is displayed by the system
Main Success Scenario	User Action System Action
	1. User will select Reports from main menu. 2. System displays the window and prompts to choose the expenses or income 3. User chooses the specific option. 4. System display

		graphically.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.11: Use Case view reports

2.22 View budgeting

UC-012	
USE CASE SECTION	**COMMENTS**
Use case Name	View budgeting
Scope	Everyone
Level	So that user can view budgeting.
Primary Actor	user
Preconditions	Application should be installed.
Success Guarantee	Summary is displayed by the system

Main Success Scenario	User Action	System Action
	1. User will select budgeting from main menu.	2. System can display the budget of the current month also display dept if any.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.12: Use Case View budget

2.23 View savings

UC-013	
USE CASE SECTION	**COMMENTS**
Use case Name	View savings
Scope	Everyone
Level	So that user can view savings.
Primary Actor	User
Preconditions	Application should be installed.

Success Guarantee	savings is displayed by the system	
Main Success Scenario	User Action	System Action
	1. User will select savings from main menu. 3. User can view whole saving of the current month	2. System displays the saving. .
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.13: Use Case View savings

2.24 Add laundry

UC-014	
USE CASE SECTION	**COMMENTS**
Use case Name	Add Laundry

Scope	Everyone	
Level	So that user can add laundry.	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	laundry will be added in the system	
Main Success Scenario	User Action	System Action
	1. User will select laundry from main menu. 3. User selects the add laundry. 5. User will select to specific option. 7. User will fill the form.	2. System displays the laundry window. 4. System will display option of take photo or add from gallery. 6. System will display form. 8. System adds the laundry item.
Special Requirements	There is no special requirement for this use	

	case
Frequency Of Occurrence	Medium

Table 2.14: Use Case Add laundry

2.25 Edit laundry

UC-015	
USE CASE SECTION	**COMMENTS**
Use case Name	Edit Laundry
Scope	Everyone
Level	So that user can Edit Laundry.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Income will be edited in a system
Main Success Scenario	User Action System Action

	1. User will select Laundry from main menu. 3. User selects already added Laundry and will select to edit. 5. User will enter the required data.	2. System displays the Laundry window. 4. System prompts to fill required fields. 6. System will verify the data and will save Laundry edited.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.15: Use Case Edit laundry

2.26 Delete Laundry

UC-016	
USE CASE SECTION	**COMMENTS**
Use case Name	Delete Laundry
Scope	Everyone

Level	So that user can delete Laundry.	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	Laundry will be deleted in the system	
Main Success Scenario	User Action	System Action
	1. User will select Laundry from main menu. 3. User selects the already present Laundry. 5. User will select to delete Laundry.	2. System displays the Laundry window. 4. System will display detail of Laundry selected. 8. System will delete the selected Laundry.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.16: Use Case Delete laundry

2.27 View Laundry

UC-017	
USE CASE SECTION	**COMMENTS**
Use case Name	View Laundry
Scope	Everyone
Level	So that user cans View Laundry.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Laundry will be View in a system
Main Success Scenario	User Action / System Action
	1. User will select Laundry from main menu. 2. System displays the Laundry window. 3. User selects already added Laundry and will select to View. 4. System displays Laundry items.
Special Requirements	There is no special requirement for this use

	case
Frequency Of Occurrence	Medium

Table 2.17: Use Case View laundry

2.28 Set Price

UC-018	
USE CASE SECTION	**COMMENTS**
Use case Name	Set Price
Scope	Everyone
Level	So that user can set price.
Primary Actor	User
Preconditions	Application should be installed.
Success Guarantee	Price will be added in a system
Main Success Scenario	User Action System Action

	1. User will select set price from main menu. 3. User set price	2. System displays the set price window along with categories of laundry. 4. System will save the data.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.18: Use Case Set prices

2.29 Add Relatives

UC-019	
USE CASE SECTION	**COMMENTS**
Use case Name	Add relatives

Scope	Everyone	
Level	So that user can add relatives	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	Relatives will be added in a system	
Main Success Scenario	User Action	System Action
	1. User will select meal today from main menu. 3. User will select the add relatives. 5. User will enter the required data.	2. System displays different options 4. System prompts to fill required fields for example name, phone number, relation and gender. 6. System will added and save it into data base.
Special Requirements	There is no special requirement for this use case	

Frequency Of Occurrence	Medium

Table 2.17: Use Case Add relatives

2.30 Create menu

UC-018		
USE CASE SECTION	**COMMENTS**	
Use case Name	Create menu	
Scope	Everyone	
Level	So that user can add dishes of weak	
Primary Actor	User	
Preconditions	Application should be installed.	
Success Guarantee	Dishes and days will be added in a system	
Main Success Scenario	User Action	System Action
	1. User will select meal today from main menu.	
		2. System displays different options
	3. User will select the Create menu.	
		4. System prompts to fill required fields for

	5. User will enter the required data.	dishes name and days of weak. 6. System will added and save it into data base.
Special Requirements	There is no special requirement for this use case	
Frequency Of Occurrence	Medium	

Table 2.18: Use Case Create menu

2.31 Send messages

UC-019	
USE CASE SECTION	**COMMENTS**
Use case Name	Send messages
Scope	Everyone
Level	So that user can Send messages and decide

	dish of the day.	
Primary Actor	User	
Preconditions	Relatives already present in the database to which messages should deliver.	
Success Guarantee	Messages should be successfully send to relatives which are already added in the database.	
Main Success Scenario	User Action	System Action
	1. User will select meal today from main menu. 3. User will select the Send messages. 5. User will select the required data.	2. System displays different options 4. System prompts to select the relatives and dish of the day 6. System will send the message successfully to the specific selected relatives.
Special Requirements	There is no special requirement for this use	

	case
Frequency Of Occurrence	Medium

Table 2.19: Use Case Send messages

2.32 System Sequence Diagrams

System sequence diagram (SSD) is a sequence diagram that shows, for a particular scenario of a use case, the event that external actors generate their orders, and possible inter-system events. System Sequence Diagrams for "Home Ministry" are shown below:

2.32.1 Add income

The user selects Add income option. System prompts details. User enters data. System saves changes further shown in figure 2.2:

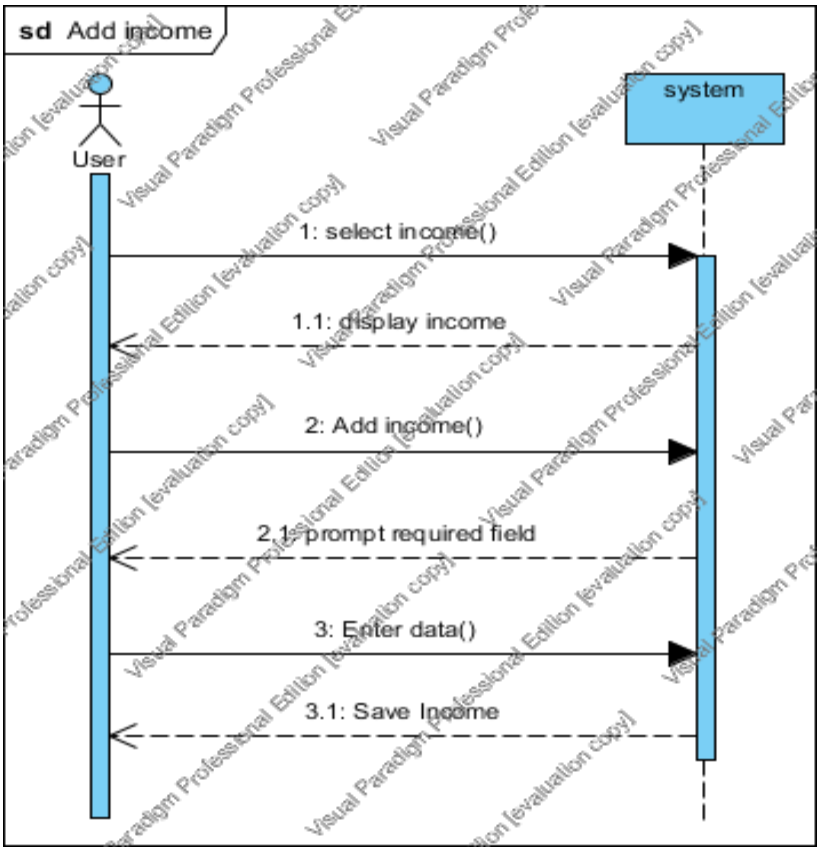


Figure 2.2: System Sequence for Add income

2.32.2 Edit income

User selects option income. System display income. User select edit income. System prompts to enter some data. User enters data .system save it into data base .As given in the figure 2.3:

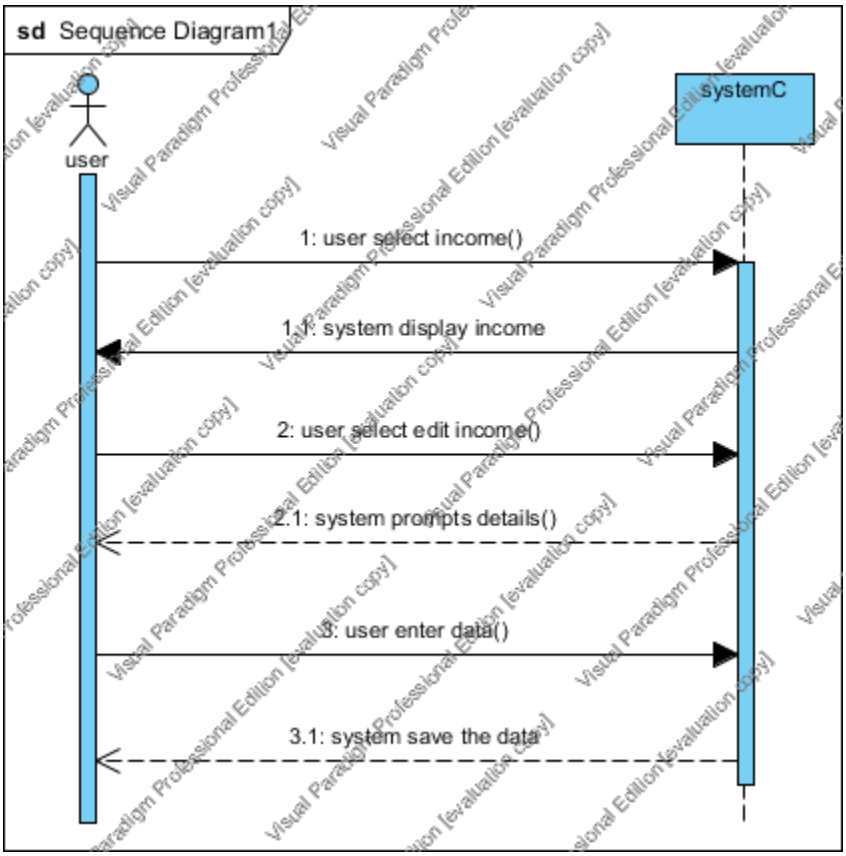


Figure 2.3: System Sequence for edit income

2.32.3 Delete income

User selects income. System display income. User select delete income option. System prompts to select some data and perform deletion on that data. User selects data and performs deletion. As given in the figure 2.4:

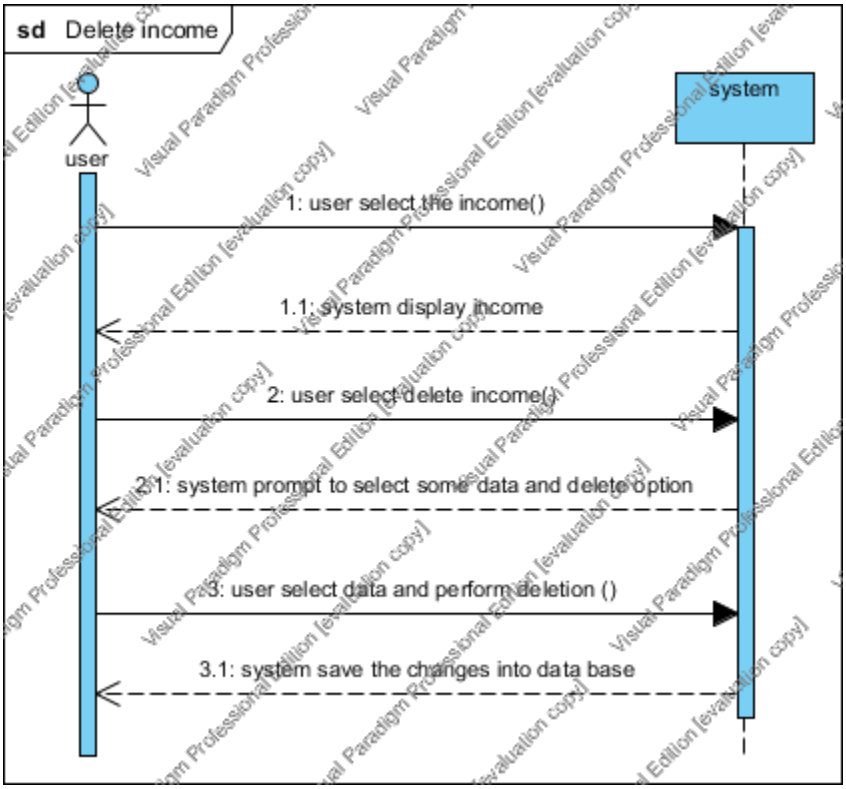


Figure 2.4: System Sequence Diagram deletes income

2.32.4 View income

User selects view income. System display view income.User select exit option. Systems will exit. As given in the figure 2.5:

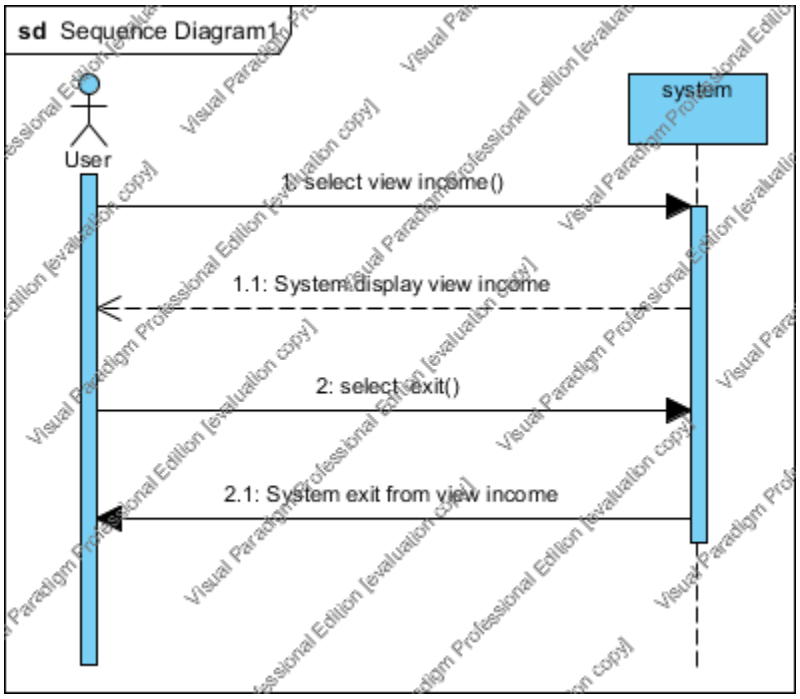


Figure 2.5: System Sequence Diagram for view income

2.32.5 Add expenses

User selects Add expenses option. System prompts details. User enters data. System saves changes. As given in the figure 2.6:

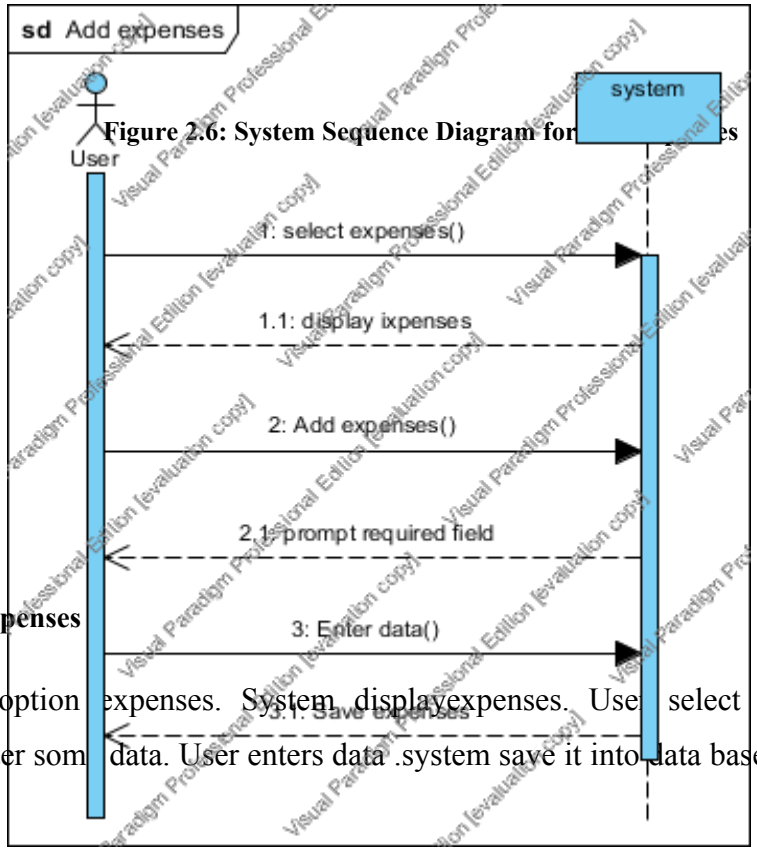


Figure 2.6: System Sequence Diagram for

2.32.6 Edit expenses

User selects option expenses. System displayexpenses. User select edit expenses. System prompts to enter some data. User enters data .system save it into data base. As given in the figure 2.7:

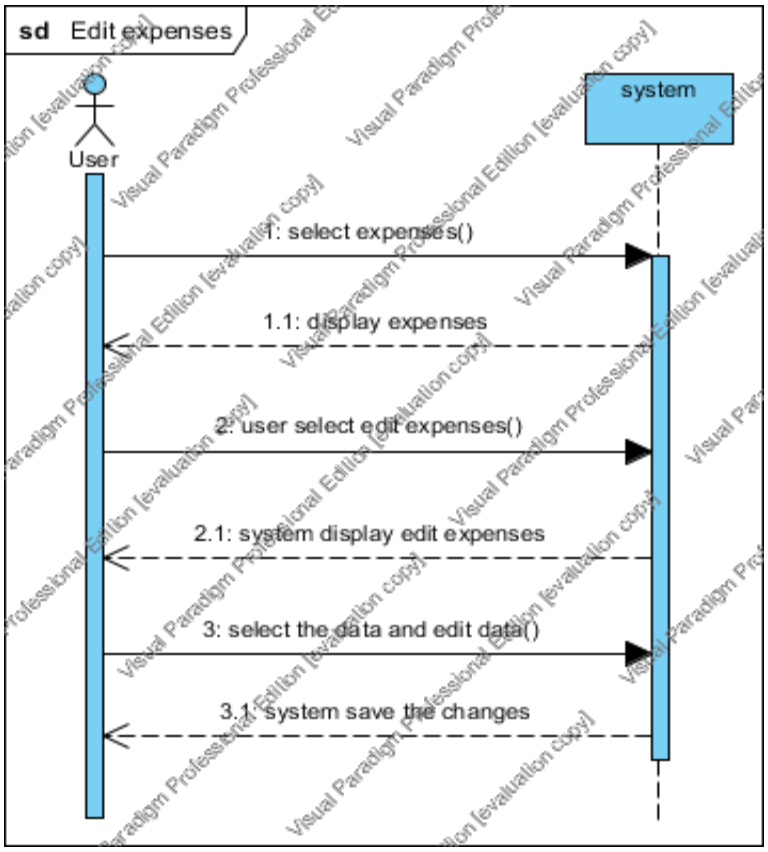


Figure 2.7: System Sequence Diagram for Edit expenses

2.32.7 Delete expenses

User selects expenses. System display expenses. User select delete expenses option. System prompts to select some data and perform deletion on that data. User selects data and performs deletion. As given in the figure 2.8:

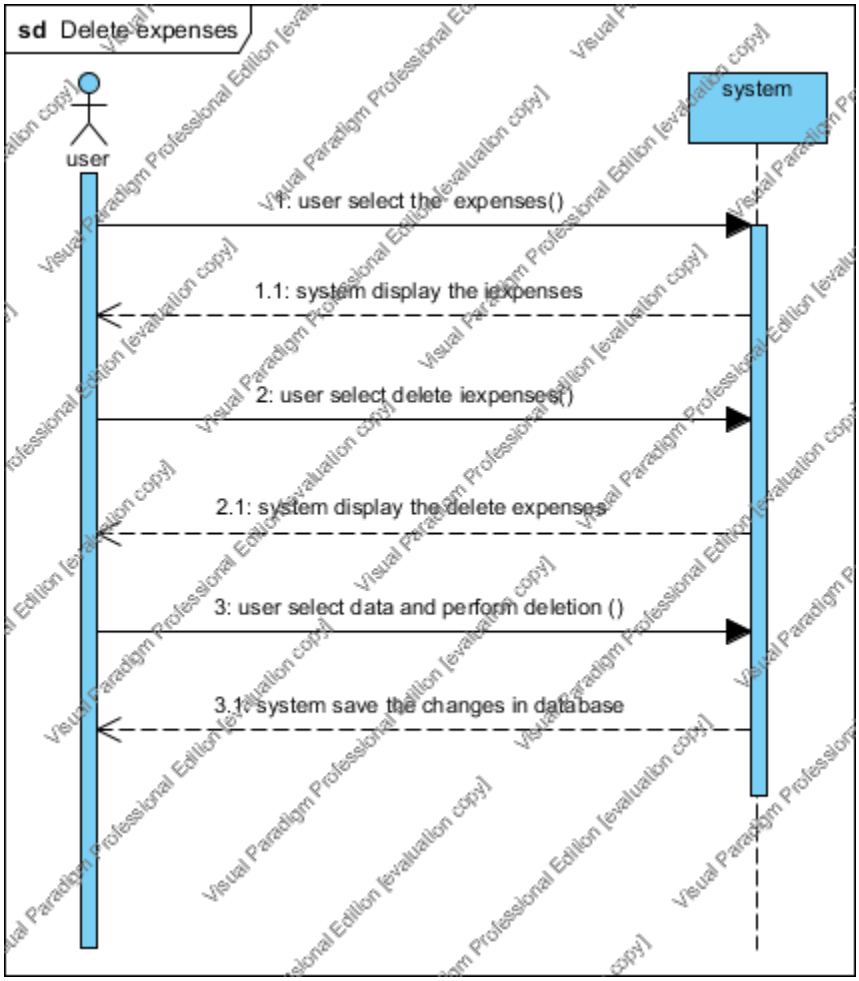


Figure 2.8: System Sequence Diagram for Delete expenses

2.32.8 Add laundry

Use select laundry. System display laundry. User select add laundry. System display camera and gallery icons. User selects the option and takes pictures. System display categories of laundry. User selects the category. System displays the price of selected category and save the data. As given in the figure 2.9:

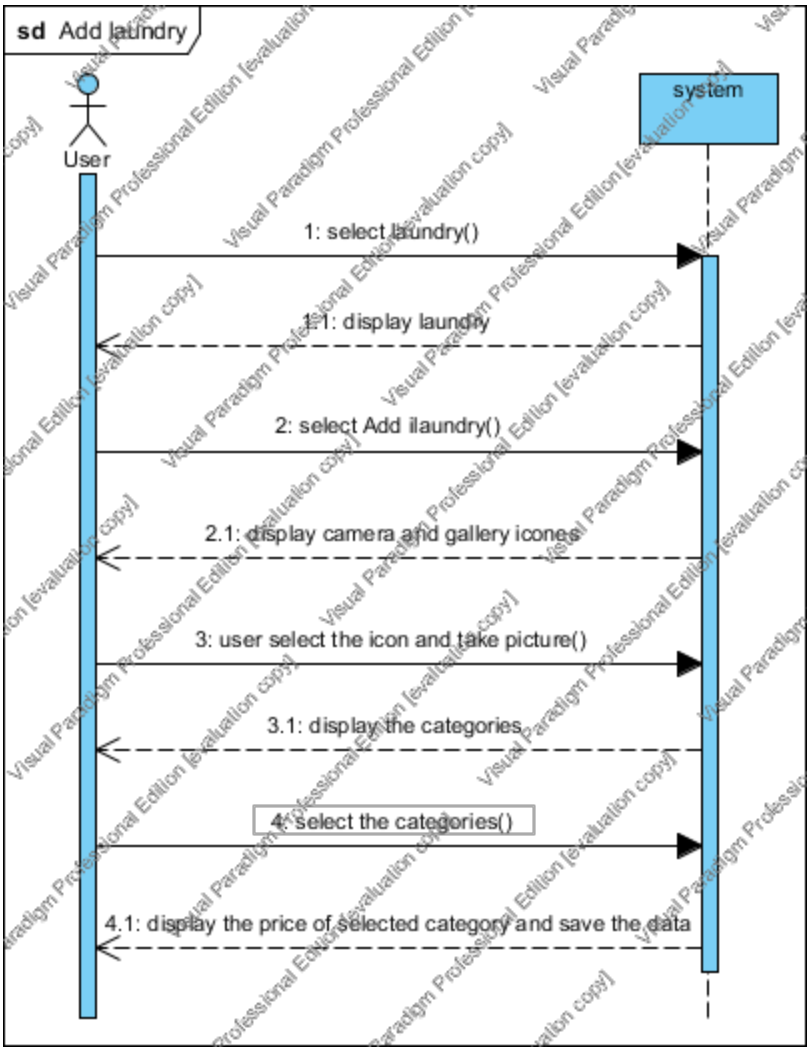


Figure 2.9: System Sequence Diagram for Add laundry

2.32.9 Edit laundry

User can select laundry. System display laundry. User select edit laundry. System displays it. User selects the data and edits data. System saves the changes into data base. As given in the figure 2.10:

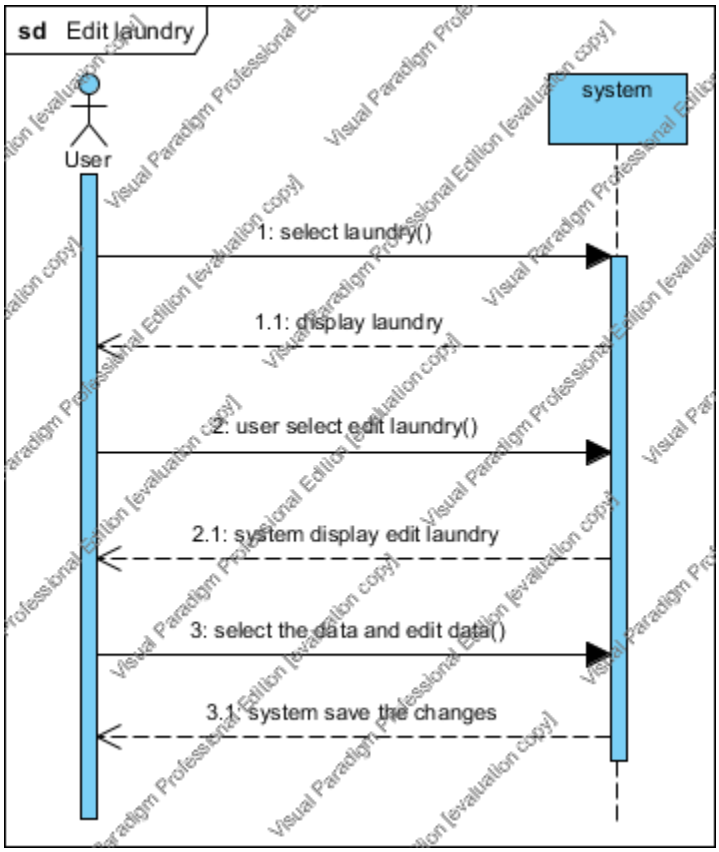


Figure 2.10: System Sequence Diagram for Edit laundry

2.32.10 Delete laundry

User selects laundry. System display laundry. User select delete laundry option. System prompts to select some data and perform deletion on that data. User selects data and performs deletion. As given in the figure 2.11:

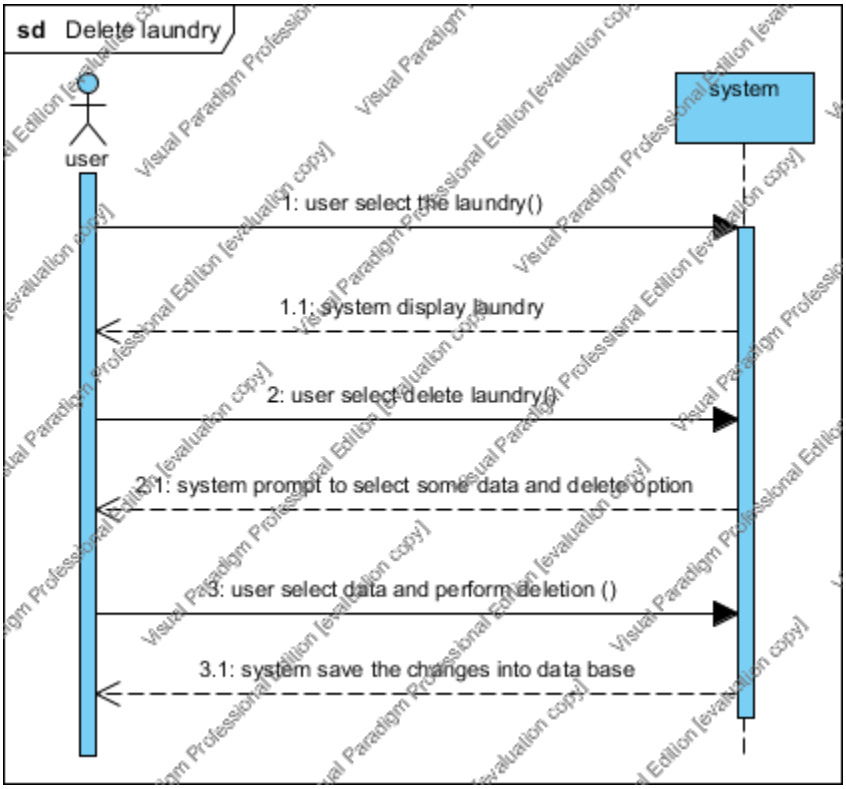


Figure 2.11: System Sequence Diagram for Delete laundry

2.32.11 View summary

User can select view summary. System prompts to enter month or year. User enters data. System display summary of income and expenses of entire month. As given in the figure 2.12:

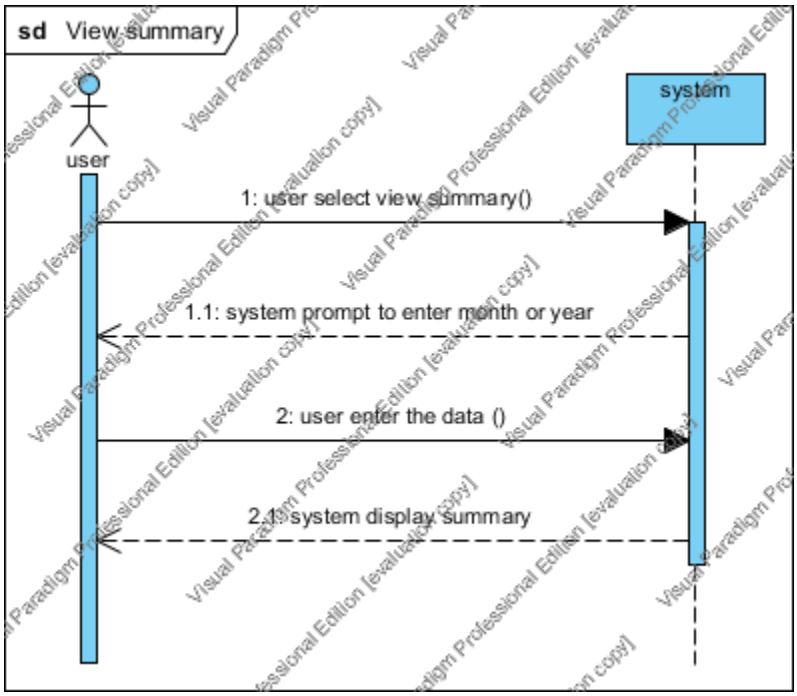


Figure 2.12: System Sequence Diagram for View summary

2.32.12 View savings

User selects view savings. System displays the savings of current month if nay. As given in the figure 2.13:

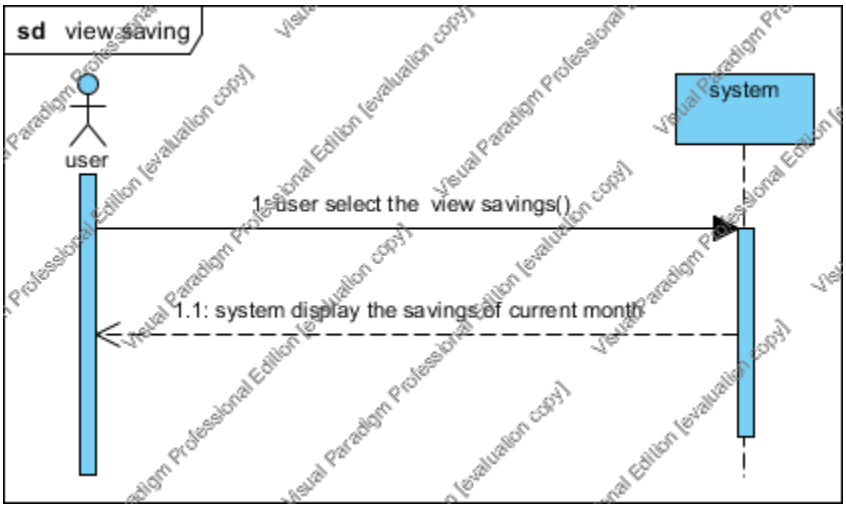


Figure 2.13: System Sequence Diagram for View savings

2.32.13 View budget

User selects view budget. System calculates the budget and display budget and dept if any. As given in the figure 2.14:

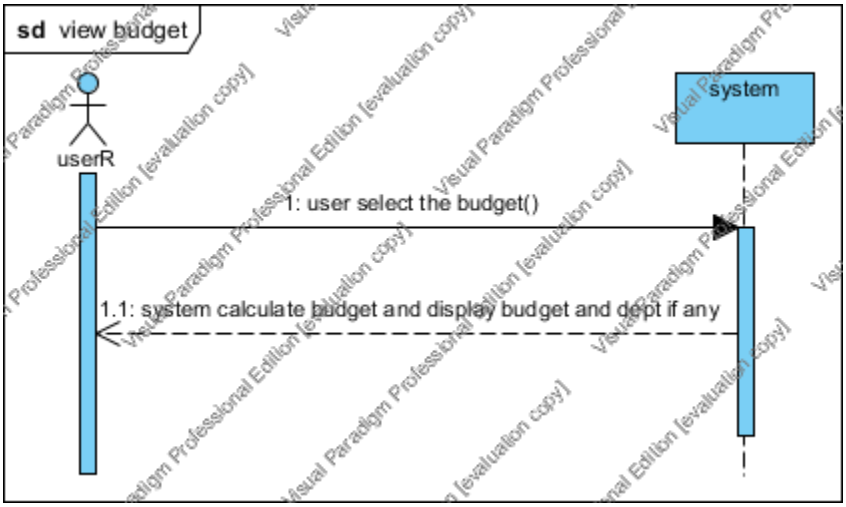


Figure 2.14: System Sequence Diagram for View budget

2.32.14 View reports

User select view reports. System displays the option of income and expenses. User selects the option. System displays the report graphically of current month. As given in the figure 2.15:

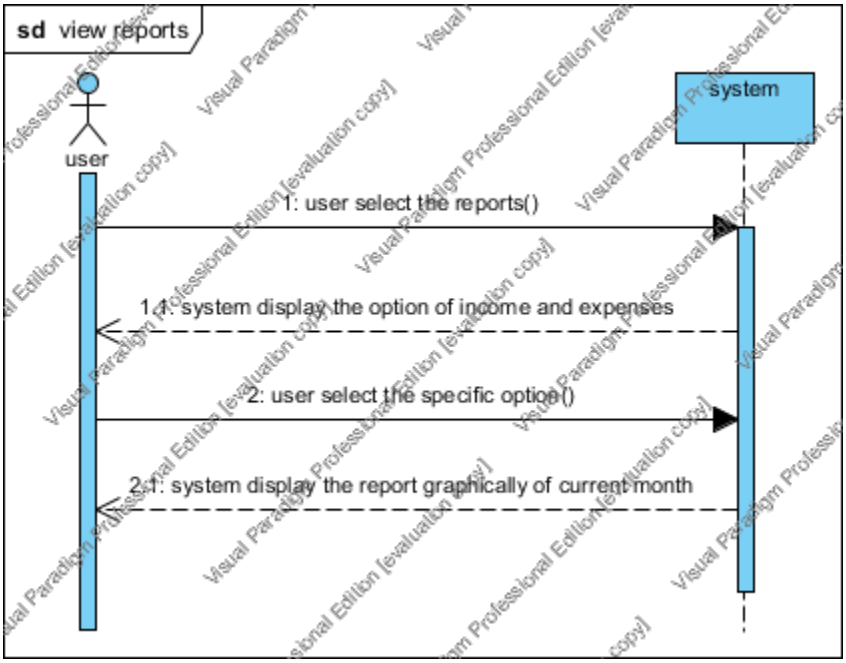


Figure 2.15: System Sequence Diagram for View budget

2.32.15 Add relative

User select add relative. System prompts to enter data. User enters data. System display adds option. User selects the option. System saves the data into data base.As given in the figure 2.16:

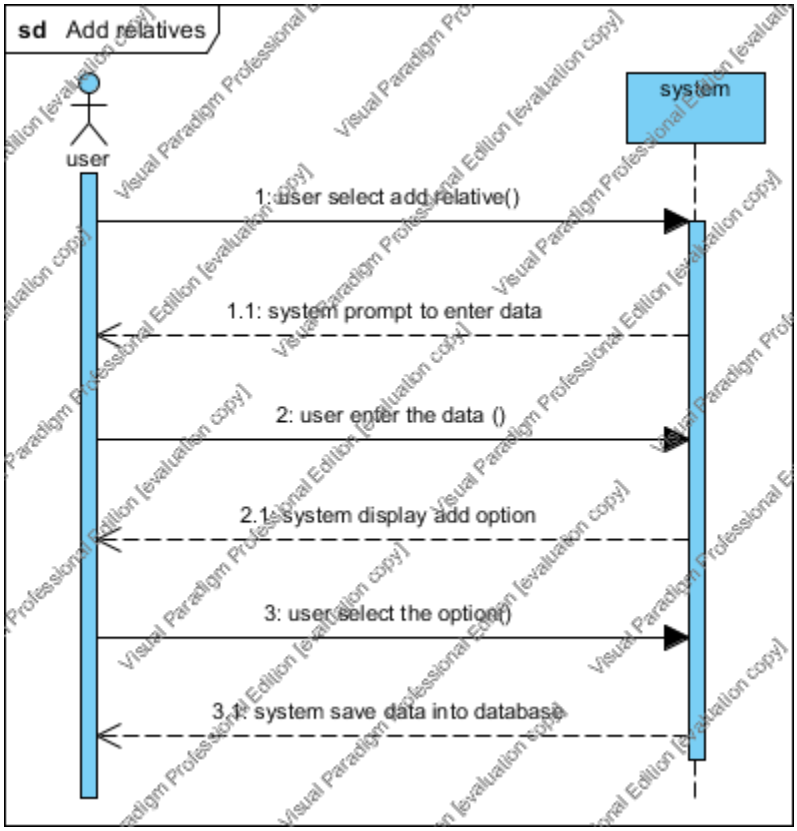


Figure 2.16: System Sequence Diagram for Add relatives

2.32.16 Send messages

User can select meal today. System display options. User select add dishes. System displays dishes name and days. User selects the option. System displays the option of select relatives and sends messages. User selects relatives and sends message option. System saves data into data base. As given in the figure 2.17:

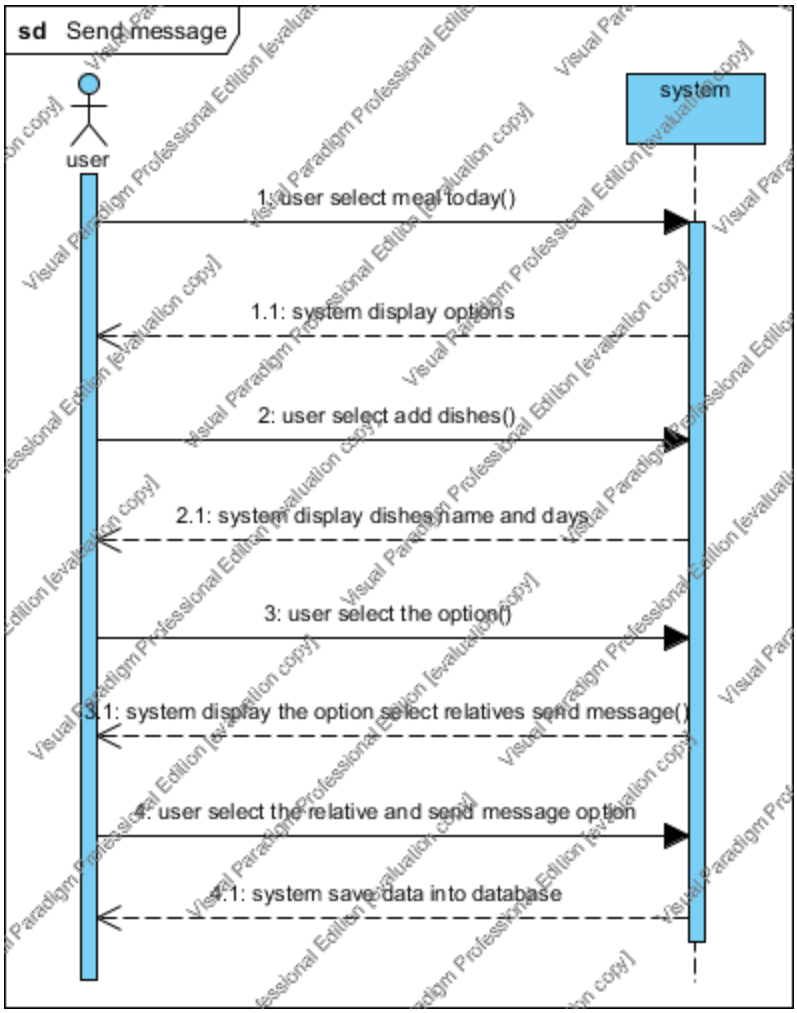


Figure 2.17: System Sequence Diagram for Add relatives

2.32.17 Set menu

User can select meal today. System display options. User select add dishes. System displays dishes name and days. User selects the option. System saves data into data base. As given in the figure 2.18:

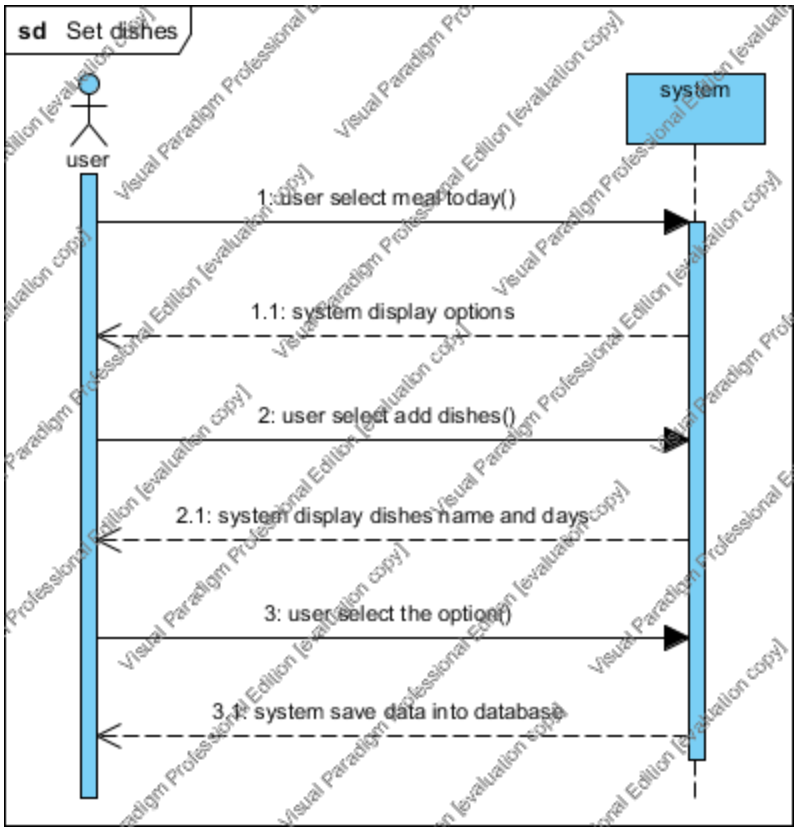


Figure 2.18: System Sequence Diagram for Set dishes

2.33 Domain Model

In domain modeling class diagrams, visibility defines whether attributes and operations of specific classes can be seen and used by other classes. The Domain Model of The Home Ministry is shown in figure 2.14:

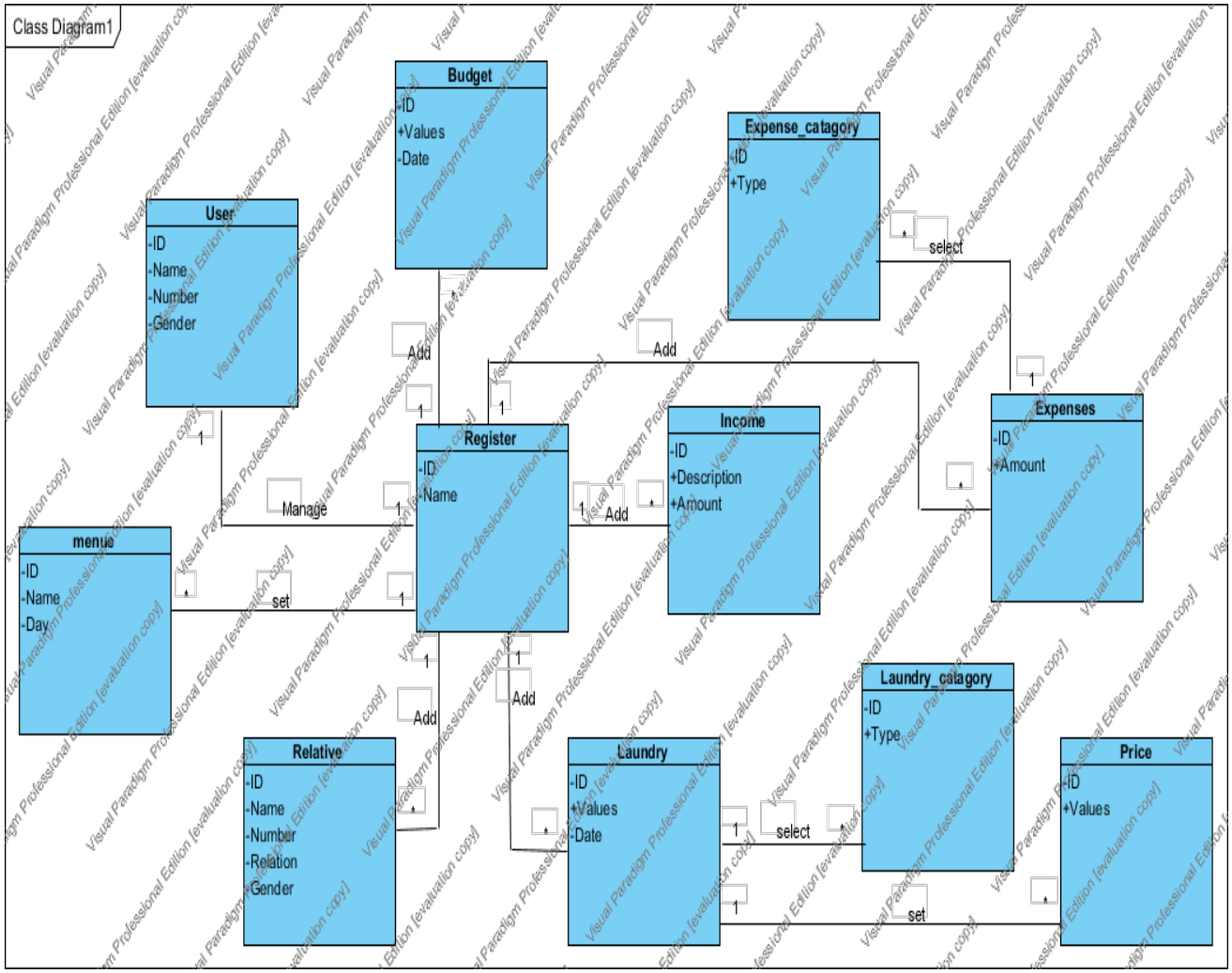


Figure 2.14: Domain Model of the Home Ministry

CHAPTER 3

SYSTEM ANALYSIS

3 System Analysis

System Analysis is a software engineering task that bridges the gap between system level requirements engineering and software design.

3.1 Specific Functional Requirement

Functional requirements of a system define the internal workings of the software. The functional requirements of The Home Ministry are as follows:

3.1.1 User Authentication

User opens the application; firstly she/he has to sign in. If she/he is not registered then she/ he has to sign up

It has three main options of budgeting, laundry, cooking.

3.1.2 Budgeting

In budgeting option user should be able to maintain income, expenses and savings.

- **Add income**
- **Delete income**
- **View income**
- **Edit income**
- **Add expenses**
- **View expenses**
- **Edit expenses**
- **Delete expenses**
- **View savings**
- **View summary**
- **View budget**

3.1.3 Laundry

- **Add laundry**
- **Delete laundry**
- **View laundry**
- **Edit laundry**

3.1.4 Meal today

- **Add Relative**
- **Send messages**
- **Set menu**

3.2 Nonfunctional Requirements

Nonfunctional requirements are also important to be identified and fulfilled. Nonfunctional requirements of The Home Ministry are given as follows:

3.2.1 Performance Requirements

Response Time

It should be less than 10 seconds to prompt scoreboard.

Startup Time

Start Up time should be less than 20 seconds.

Exit Time

Exit time should beless than 10 seconds

Graphical User Interface Requirements

User Interface should be captivating and attractive.

3.2.2 Portability Machine

The recommended phone should have android operating system.

3.2.3 Language

The system should support English language only.

3.2.4 Availability

The System availability and accessibility should be 24/7.

3.2.5 Continuity

The system shall be on till user decides to exit.

3.2.6 Reliability

The system should be reliable.

3.2.7 Usability

The interface should be consistent and easy to understand. User manuals should be given to user for further understanding. As the application is for people of all house hold person so it is important that GUI functionality is easily accessible.

3.3 User Characteristics

To useThe Home Ministry, user must be able to use Android phone. User can be install this application on their phone.

CHAPTER 4

SYSTEM DESIGN

4 System Design

This chapter illustrates the diagrams which show the relationship between the classes, objects and their attributes and the functionality performed by the classes.

4.1 Class Diagram

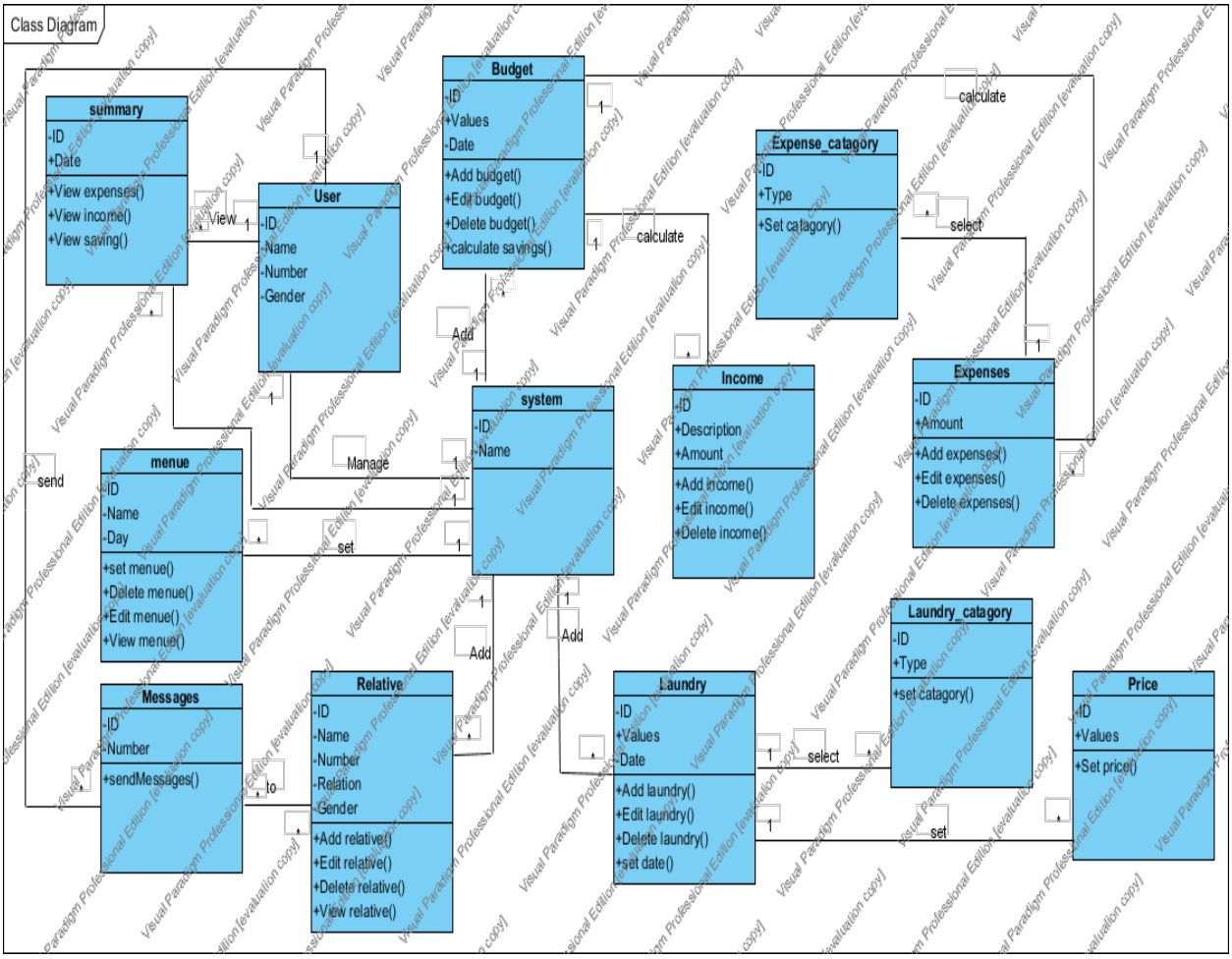


Figure 4.1: Class Diagram of Home Ministry

CHAPTER 5

IMPLEMENTATION

5 Implementation:

Implementation starts when design of the proposed system is complete. Design of the system is converted from paper design to actual code via some programming language. This chapter will discuss the programming code of The Home Ministry. Logical thinking and coding is required at the development of the application. A lot of research has been done to enhance the efficiency while coding.

5.1 Software Development Lifecycle Model:

The methodology used for the development lifecycle is kind of like Agile Model. The purpose for choosing agile is that Agile welcomes late changes and there are no stern limitations of completing the requirements and design flow before coding. It develops software iteratively with a heavy emphasis on construction activities. Produce incremental product rapidly. Progress measured by running tested software. It provides early and regular process development.
Successful implementation needs some tools. Following tools are used to implement this project.

5.2 Software Development Tool:

The tool used for implementation is:

- Android

Object oriented and java concepts have been used to manage the logic behind each line of code.

5.3 Pseudo codes:

The coding phase of the project includes the basic main algorithms of the project and their brief description to better explain them.

5.3.1 Pseudo code Login

```
GET Username
GET Password
IF (Username==Entered Username && Password==Entered Password) THEN
Login Successful
ELSE
Login Failed
ENDIF
```

5.3.2 Pseudo code Budgeting

```
SELECT budget
User selects income, expenses, saving, summary, reports
If user select income
Case 1: Add income
Enter income
Income is added successfully
Case 2: Delete income
Select income
Perform deletion
User deletes income successfully
```

Case 3: Edit income

Select edit income

User edit income

Else if user select expenses

Case 1: Add expenses

Enter expenses

Expense is added successfully

Case 2: Delete expenses

Select expenses

Perform deletion

User deletes expenses successfully

Case 3: Edit expenses

Select edit expenses

User edit expenses

5.3.3 Pseudo code laundry

If user select laundry management

Case 1: Add laundry

Choose category

Take picture or take from gallery to insert laundry

Price display according to category

User is added picture with price successfully

Case 2: Delete laundry

Select laundry

Perform deletion

User deletes laundry successfully

Case 3: Edit laundry

Select edit laundry

User edit laundry

Case 4: View laundry

Select view laundry

User view laundry in grid view

5.3.4 Pseudo code set menu

IF user select set menu

User enter dish name and day

Dishes added according to the days of whole week

ELSE IF Select edit dishes

User selects the dishes

User edits dishes successfully

ELSE IF Select delete dishes

User select dishes

User performs deletion on selected items.

5.3.5 Pseudo code Add relatives

Case 1: Add relative

Enter name, number, Gender, relation

Relative added successfully

Case 2: Delete relative

Select relative

Perform deletion

User deletes income successfully

5.3.6 Pseudo code send messages

Selects send messages

If relatives exist

Select relatives

And send message

Else

Error messages display on screen add relatives

5.3.7 Set prices

Select set prices

System displays categories

User set prices on the basis of category

5.3.8 View savings

Select view savings

All savings display

5.3.9 View reports

Select view reports

Choose expenses and income

User choose the option

Reports display graphically according to entered choice

5.3.10 View summary

Select expenses and income

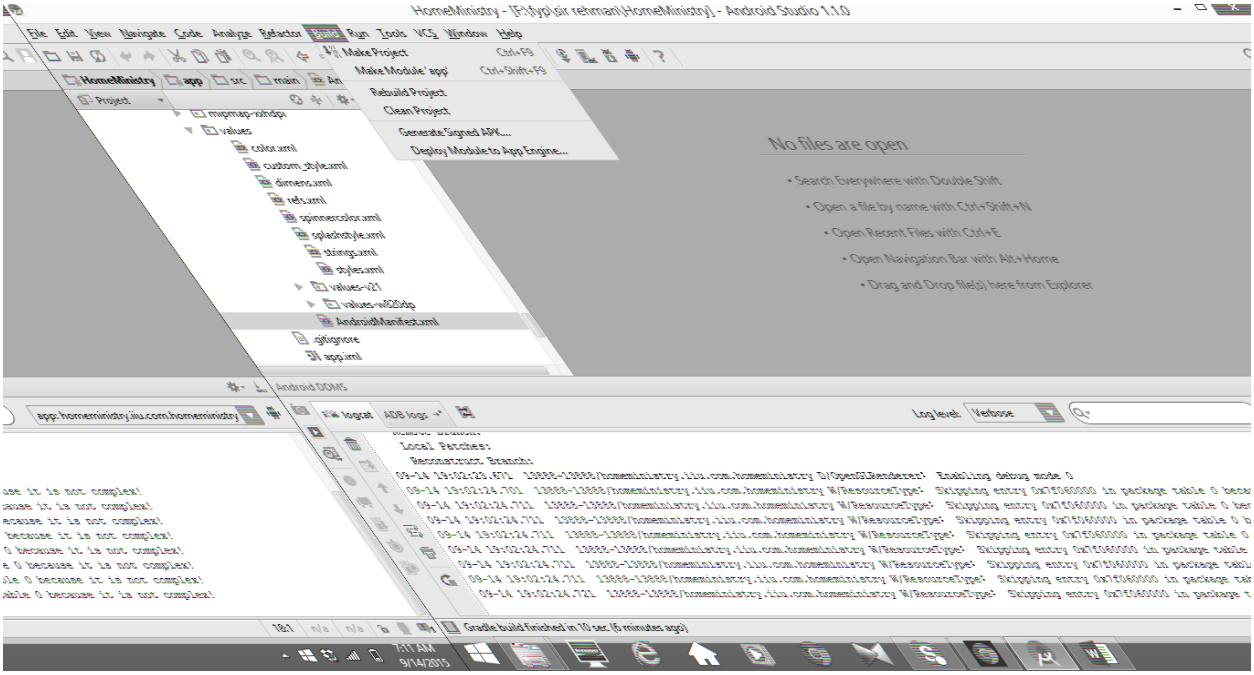
Entered day, month, and year

Display summary of expenses and income on the basis of entered date

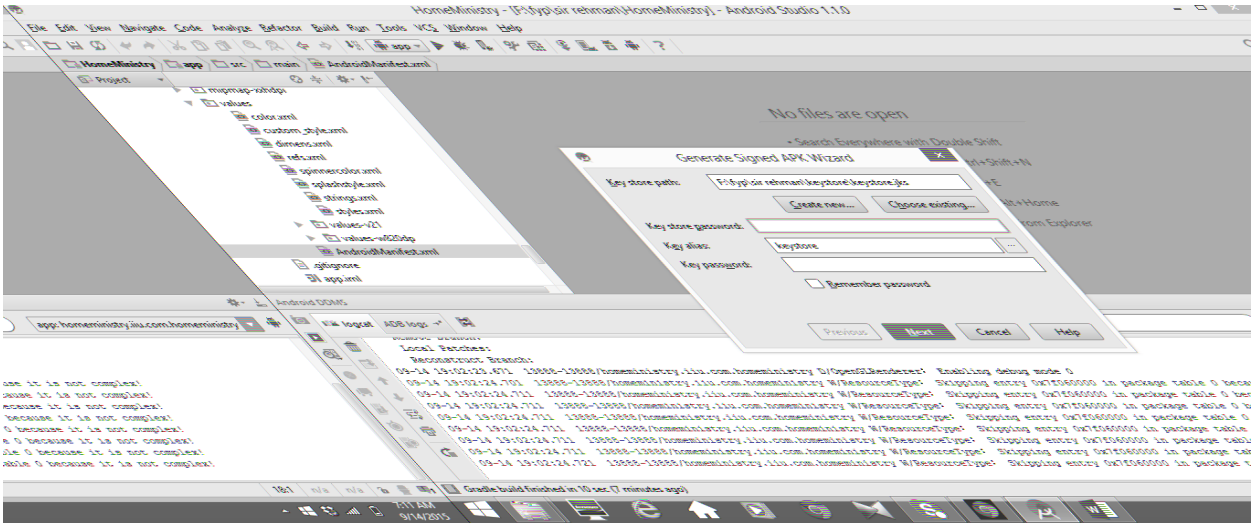
5.4 Deployment Phase

Deployment of application is mainly on Android phone Deployment Diagram

Figure 5.1a.b illustrates the deployment diagram of this project.

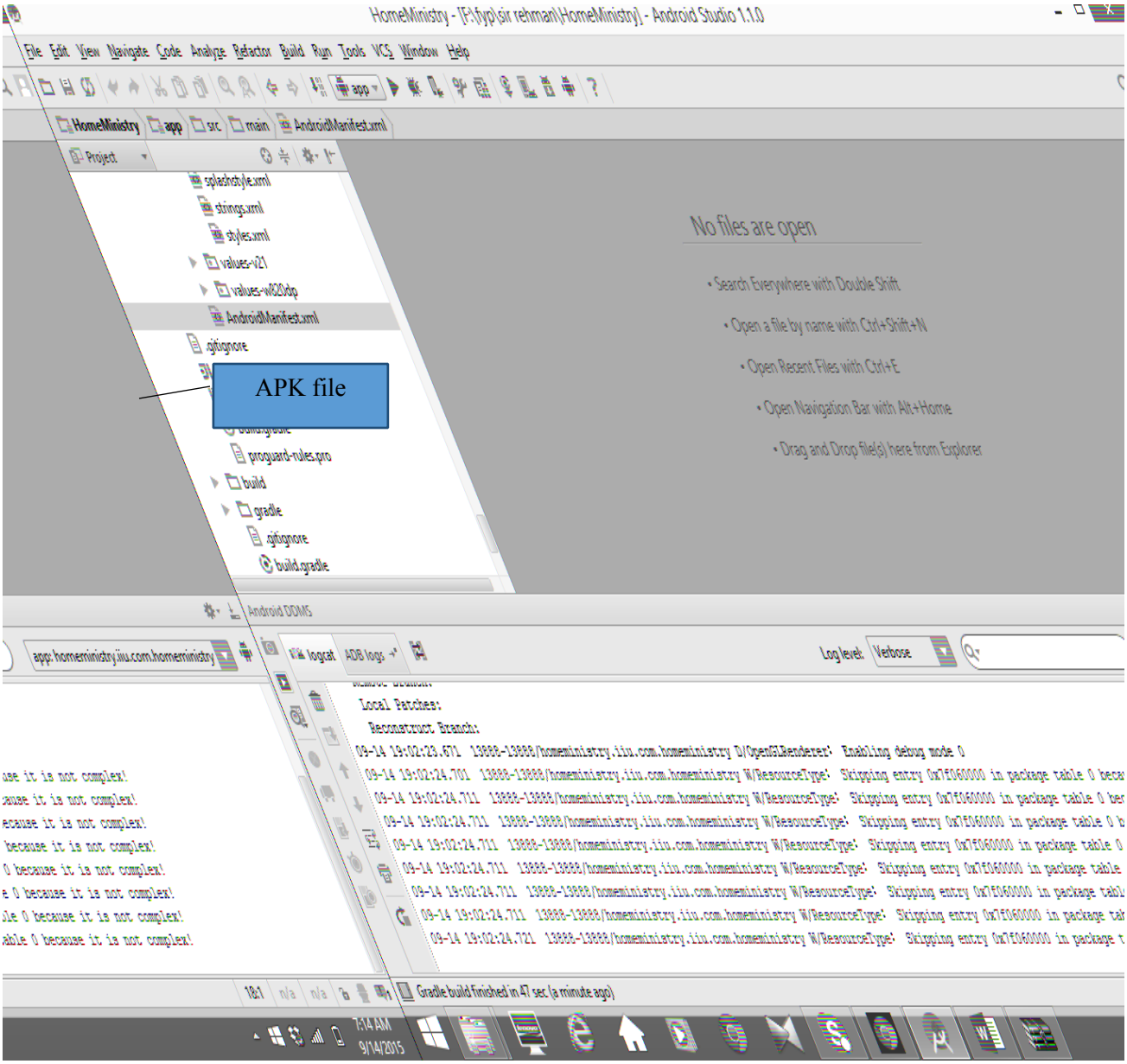


1. First of all we go to build in android studio to generate APK file of our application.



2. Then we set its location.

Figure 5.1a: Deployment diagram of The Home Ministry


HomeMinistry - [F:\fyp\sir rehman\HomeMinistry] - Android Studio 1.1.0
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
HomeMinistry app src main AndroidManifest.xml
Project
splashstyle.xml
strings.xml
styles.xml
values-v21
values-w820dp
AndroidManifest.xml
gitignore
APK file
proguard-rules.pro
build
gradle
gitignore
build.gradle
No files are open
• Search Everywhere with Double Shift
• Open a file by name with Ctrl+Shift+N
• Open Recent Files with Ctrl+E
• Open Navigation Bar with Alt+Home
• Drag and Drop file(s) here from Explorer
Android DDMS
app.homeministry.iiu.com.homeministry
logcat
ADB logs
Log level: Verbose
Local Patches:
Reconstruct Branch:
09-14 19:02:23.671 13888-13888/homeministry.iiu.com.homeministry D/OpenGLRenderer: Enabling debug mode 0
09-14 19:02:24.701 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package table 0 beca
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package table 0 bec
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package table 0 b
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package table 0
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package table
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package tabl
09-14 19:02:24.711 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package tab
09-14 19:02:24.721 13888-13888/homeministry.iiu.com.homeministry W/ResourceType: Skipping entry 0x7f060000 in package t
use it is not complex!
cause it is not complex!
ecause it is not complex!
because it is not complex!
0 because it is not complex!
e 0 because it is not complex!
le 0 because it is not complex!
able 0 because it is not complex!
18:1
Gradle build finished in 47 sec (a minute ago)
7:14 AM
9/14/2015

3. APK is released

Figure 5.1b: Deployment diagram of The Home Ministry

CHAPTER 6

TESTING

6 Testing

Test cases are written against each and every mentioned use case.

6.1 Test Cases

Following test cases conducted for The Home Ministry System.

6.1.1 Login

Test Case ID:	**TC-001**	
Purpose	Login into the application	
Functionality to be Tested:	Logged in Successfully	
Actor:	User	
Pre Conditions:	Application should be installed.	
Test Data:	Login={Valid= if username and password entered is correct; Invalid= username and password entered is incorrect}	
Post Conditions:	Logged in successfully.	
Steps:	1. Enter username. 2. Enter password 3. Click on the Login button	
Test Results:	**Basic Flow**	**Expected Result**
	Click on the Login button.	Menu of application is opened and user performs the required action.

Test Case Status:	**Fail**

Table 6.1: Test case login

6.1.2 Login

Test Case ID:	**TC-002**	
Purpose	Login into the application	
Functionality to be Tested:	Logged in Successfully	
Actor:	User	
Pre Conditions:	Application should be installed.	
Test Data:	Login={Valid= if username and password entered is correct; Invalid= username and password entered is incorrect}	
Post Conditions:	Logged in successfully.	
Steps:	1. Enter username. 2. Enter password 3. Click on the Login button	
Test Results:	**Basic Flow**	**Expected Result**
	Click on the Login button.	Menu of application is opened and user performs the required action.
Test Case Status:	**Pass**	

Table 6.2: Test case signup

6.1.3 Sign up

Test Case ID:	**TC-003**	
Purpose	Register a user	
Functionality to be Tested:	User is registered successfully	
Actor:	User	
Pre Conditions:	Application should be installed.	
Test Data:	Register={Valid data entered is correct; Invalid= data is incorrect}	
Post Conditions:	Registered successfully.	
Steps:	1. Enter username. 2. Enter Phone number 3. Click on the Register button	
Test Results:	**Basic Flow**	**Expected Result**
	Click on the Register button.	Menu of application is opened and user performs the required action.
Test Case Status:	**Fail**	

Table 6.3: Test case 2 signup

6.1.4 Sign up

Test Case ID:	**TC-004**	
Purpose	Register a user	
Functionality to be Tested:	User is registered successfully	
Actor:	User	
Pre Conditions:	Application should be installed.	
Test Data:	Register={Valid data entered is correct; Invalid= data is incorrect}	
Post Conditions:	Registered successfully.	
Steps:	1. Enter username. 2. Enter Phone number 3. Click on the Register button	
Test Results:	**Basic Flow**	**Expected Result**
	Click on the Register button.	Menu of application is opened and user performs the required action.
Test Case Status:	**Pass**	

Table 6.4: Test case 3 signup

6.1.5 Add income

Test Case ID:	**TC-009**	
Associated Use Case:	Add income	
Purpose	To add income and save into database	
Functionality to be Tested:	Income is added Successfully.	
Actor:	User	
Pre Conditions:	User select the add Income option	
Test Data:	Add income={Valid income is added successfully; Invalid=income is not added}	
Post Conditions:	Income is added successfully.	
Steps:	1. Go to add Income 2. Enter income 3. Save into database	
Test Results:	**Basic Flow** user select the budget and go to add income	**Expected Result** Income is added successfully
Test Case Status:	**Fail**	

Table 6.5: Test case Add income

6.1.6 View income

Test Case ID:	**TC-010**	
Associated Use Case:	view income	
Purpose	View income and it will be successfully inserted and save into database.	
Functionality to be Tested:	Income is viewed Successfully.	
Actor:	User	
Pre Conditions:	User select view income option.	
Test Data:	view income={Valid income is viewed successfully; Invalid=income is not viewed}	
Post Conditions:	Income is viewed successfully.	
Steps:	1. Go to viewed income 2. Viewed income.	
Test Results:	**Basic Flow** Income is viewed	**Expected Result** Income is viewed successfully
Test Case Status:	**Pass**	

Table 6.6: Test case view income

6.1.7 Delete income

Test Case ID:	**TC-010**	
Associated Use Case:	Delete income	
Purpose	Delete income and it will be successfully deleted and changes save into database.	
Functionality to be Tested:	Income is deleted Successfully.	
Actor:	User	
Pre Conditions:	User select delete income option.	
Test Data:	Delete income={selected income is deleted successfully; Invalid=selected income is not deleted}	
Post Conditions:	Income is deleted successfully.	
Steps:	1. Go to delete income 2. Select the income. 3. Perform deletion on selection 4. Save changes into database	
Test Results:	**Basic Flow** Income is deleted and saves changes to database.	**Expected Result** Income is deleted successfully
Test Case Status:	**Pass**	

Table 6.7: Test case Delete income

6.1.8 Add expenses

Test Case ID:	**TC-009**	
Associated Use Case:	Adding expenses	
Purpose	To add expenses and save into database	
Functionality to be Tested:	Expenses are added Successfully.	
Actor:	User	
Pre Conditions:	User select the add expenses option	
Test Data:	Add expenses ={Valid expenses is added successfully; Invalid= expenses is not added}	
Post Conditions:	Expenses are added successfully.	
Steps:	1. Go to add expenses 2. Enter expenses 3. Save into database	
Test Results:	**Basic Flow** user select the budget and go to add expenses	**Expected Result** expenses is added un successfully
Test Case Status:	**Fail**	

Table 6.7: Test case 1 Add expenses

6.1.9 Add expenses

Test Case ID:	**TC-009**

Associated Use Case:	Add expenses	
Purpose	To add expenses and save into database	
Functionality to be Tested:	Expenses are added Successfully.	
Actor:	User	
Pre Conditions:	User select the add expenses option	
Test Data:	Add expenses ={Valid expenses is added successfully; Invalid= expenses is not added}	
Post Conditions:	Expenses are added successfully.	
Steps:	1. Go to add expenses 2. Enter expenses 3. Save into database	
Test Results:	**Basic Flow** user select the budget and go to add expenses	**Expected Result** expenses is added successfully
Test Case Status:	**pass**	

Table 6.8: Test case 2Add expenses

6.1.10 View expenses

Test Case ID:	**TC-010**	
Associated Use Case:	view expenses	
Purpose	View expenses and it will be successfully inserted and save into database.	
Functionality to be Tested:	Expenses are viewed Successfully.	
Actor:	User	
Pre Conditions:	User select view expenses option.	
Test Data:	view expenses ={Valid expenses is viewed successfully; Invalid= expenses is not correctly viewed}	
Post Conditions:	Expenses are viewed successfully.	
Steps:	1. Go to viewexpenses 2. View expenses. 3. Save into database	
Test Results:	**Basic Flow** expenses is viewed	**Expected Result** expenses is viewed successfully
Test Case Status:	**Pass**	

Table 6.9: Test case view expenses

6.1.11 Delete expenses

Test Case ID:	**TC-010**

Associated Use Case:	Delete expenses	
Purpose	Delete expenses and it will be successfully deleted and changes save into database.	
Functionality to be Tested:	Expenses are deleted Successfully.	
Actor:	User	
Pre Conditions:	User select delete expenses option.	
Test Data:	Delete expenses ={selected expenses is deleted successfully; Invalid=selected expenses is not deleted}	
Post Conditions:	Expenses are deleted successfully.	
Steps:	1. Go to delete expenses 2. Select the expenses. 3. Perform deletion on selection 4. Save changes into database	
Test Results:	**Basic Flow** Expenses are deleted and save changes to database.	**Expected Result** expenses is deleted successfully
Test Case Status:	**Pass**	

Table 6.10: Test case Delete expenses

6.1.12 Add laundry

Test Case ID:	**TC-009**

Associated Use Case:	Add laundry	
Purpose	To add laundry and save into database	
Functionality to be Tested:	Laundry added Successfully.	
Actor:	User	
Pre Conditions:	User select the add laundry option	
Test Data:	Add laundry ={Valid laundry is added successfully; Invalid= laundry is not added}	
Post Conditions:	Laundry is added successfully.	
Steps:	1. Go to add laundry 2. Take photo or add from gallery 3. Select the category of laundry 4. Add photo 5. System display price according to category 6. Save into database	
Test Results:	**Basic Flow** user select the laundry and go to add laundry	**Expected Result** laundry is added successfully
Test Case Status:	**pass**	

Table 6.11: Test case Add laundry

6.1.13 Delete laundry

Test Case ID:	**TC-010**	
Associated Use Case:	Delete laundry	
Purpose	Delete laundry and it will be successfully deleted and changes save into database.	
Functionality to be Tested:	Laundry is deleted Successfully.	
Actor:	User	
Pre Conditions:	User select delete laundry option.	
Test Data:	Delete laundry ={selected expenses is deleted successfully; Invalid=selected laundry is not deleted}	
Post Conditions:	Laundry is deleted successfully.	
Steps:	1. Go to delete laundry 2. Select the laundry. 3. Perform deletion on selection 4. Save changes into database	
Test Results:	**Basic Flow** Laundry is deleted and save changes to database.	**Expected Result** Laundry is deleted successfully
Test Case Status:	**Pass**	

Table 6.12: Test case Delete laundry

6.1.14 View laundry

Test Case ID:	**TC-010**	
Associated Use Case:	view laundry	
Purpose	View laundry and it will be successfully inserted and save into database.	
Functionality to be Tested:	Laundry is viewed Successfully.	
Actor:	User	
Pre Conditions:	User select view laundry option.	
Test Data:	view laundry ={Valid laundry is added successfully; Invalid= laundry is not added}	
Post Conditions:	Laundry is viewed successfully.	
Steps:	1. Go to viewlaundry 2. View laundry. 3. Save into database	
Test Results:	**Basic Flow** expenses is viewed	**Expected Result** expenses is viewed successfully
Test Case Status:	**Pass**	

Table 6.13: Test case view laundry

6.1.15 Add relatives

Test Case ID:	**TC-009**
Associated Use Case:	Add relative

Purpose	To add relative and save into database	
Functionality to be Tested:	Relative added Successfully.	
Actor:	User	
Pre Conditions:	User select the add relative option	
Test Data:	Add relative ={Valid relative is added successfully; Invalid=relative is not added}	
Post Conditions:	Relative is added successfully.	
Steps:	1. Go to add relative 2. Add name ,relation and phone numbers 3. Save into database	
Test Results:	**Basic Flow** user select the meal today and go to add l relative	**Expected Result** relatives are added successfully
Test Case Status:	**pass**	

Table 6.14: Test case Add relatives

6.1.16 Set menu

Test Case ID:	**TC-009**
Associated Use Case:	Set menu
Purpose	To add dishes and save into database

Functionality to be Tested:	Dishes are added Successfully.	
Actor:	User	
Pre Conditions:	User select the set menu option	
Test Data:	Add dishes ={Valid dishes and day are added successfully; Invalid= day is not added}	
Post Conditions:	Dishes and day are added successfully.	
Steps:	1. Go to set menu 2. Enter dishes and select day 3. Save into database	
Test Results:	**Basic Flow** user select the set menu and go to add dishes	**Expected Result** Dishes are added successfully
Test Case Status:	**pass**	

Table 6.15: Test case set menu

6.1.17 View summary

Test Case ID:	**TC-010**
Associated Use Case:	view summary
Purpose	View expenses, income and saving whether it was correctly save into database.

Functionality to be Tested:	Summary of income, expenses and savings are viewed Successfully.	
Actor:	User	
Pre Conditions:	User select view summary option.	
Test Data:	view summary ={Valid summary is viewed according to month successfully; Invalid= summary is not according to entered month}	
Post Conditions:	Summary is viewed successfully.	
Steps:	1. Go to view summary 2. Enter the month and year 3. View expenses, income and saving of that particular year.	
Test Results:	**Basic Flow** Summary is viewed	**Expected Result** Summary is viewed successfully
Test Case Status:	**Pass**	

Table 6.16: Test case view summary

6.1.18 Send messages

Test Case ID:	**TC-010**
Associated Use Case:	Send messages
Purpose	Messages successfully send to relatives

	number for decision of meal today	
Functionality to be Tested:	Messages successfully send to relatives on their numbers.	
Actor:	User	
Pre Conditions:	User select send messages option.	
Test Data:	view summary ={Messages are send on valid number of relatives; Invalid=numbers are invalid}	
Post Conditions:	Messages are sending successfully.	
Steps:	1. Go to meal today 2. Select the dish and day 3. Select the relatives and their numbers 4. Send messages	
Test Results:	**Basic Flow** Message sent	**Expected Result** Message is sent successfully
Test Case Status:	**Pass**	

Table 6.17: Test case send message

CHAPTER 7

CONCLUSION

7 Conclusion

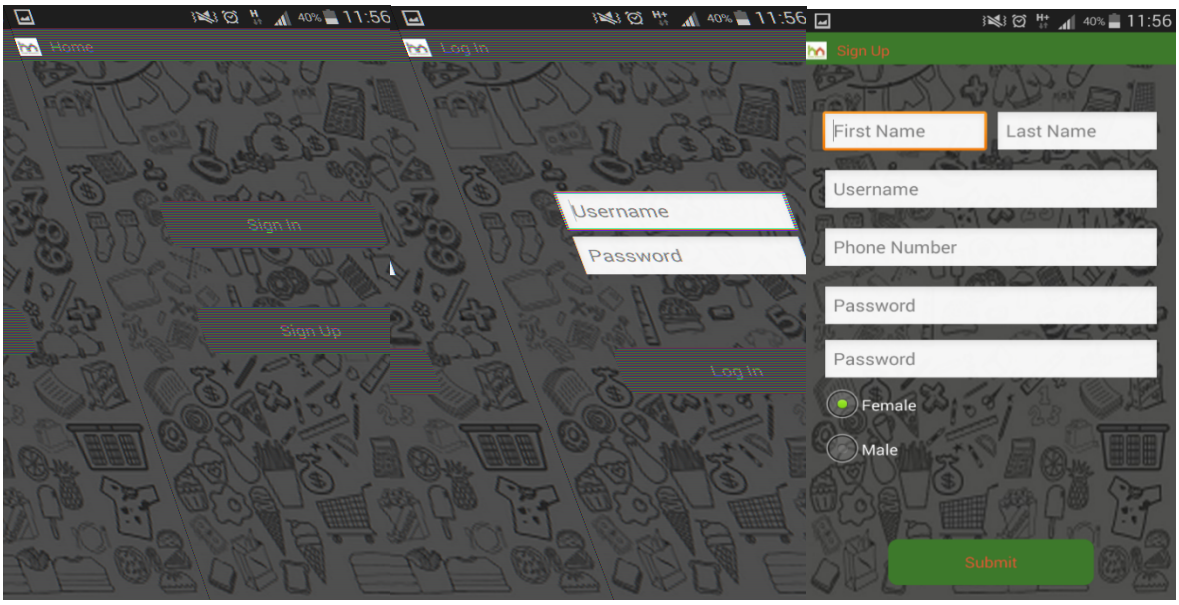
Home Ministry application is developed by using Android Studio; programming languages are Java and android.

Home Ministry has provided user many facilities. They can calculate monthly budge. They can view total monthly expenses, income and saving of the year. They can view graphical view in form of pie charts for budget. They can manage their laundry efficiently. Can get total for laundry items and the total for money. They can set weekly menu and can send message to family members to decide dish of the day. Application gives the notification of related return date for laundry and notify user when monthly expenses are more than monthly income. It is expected to be useful for the user to manage some home task.

APPENDIX A
USER MANUAL

Main Application interface:

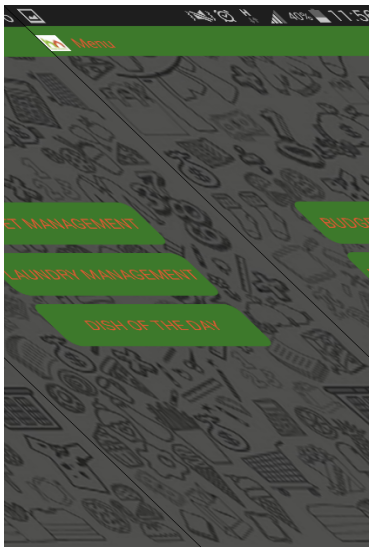
1. Login And Sign Up:



1. User clicks sign up to register itself.

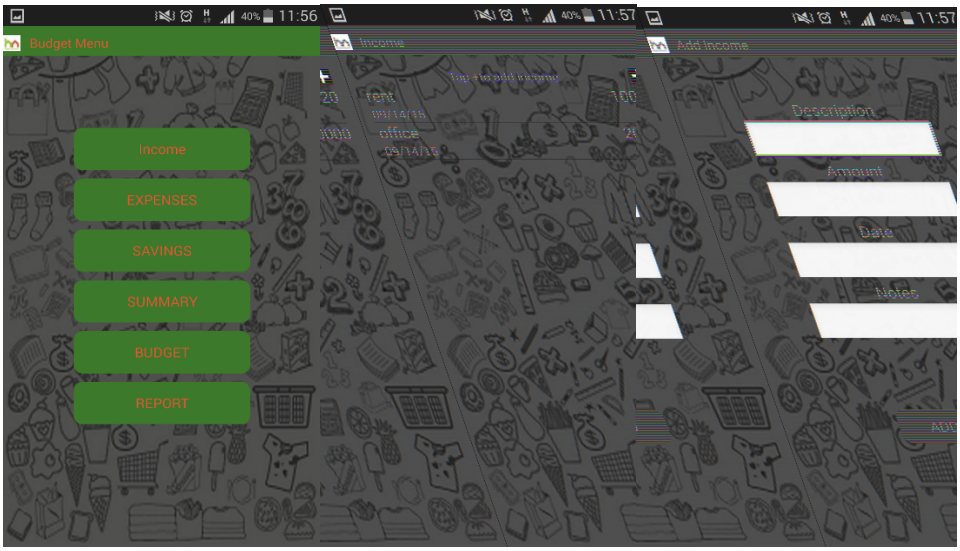
2. User click sign in then enter username and password to log in to application.

2. Menu:



1. User selects the respective buttons to manage budget, laundry or to decide dish of the day.

3. Budget Management:

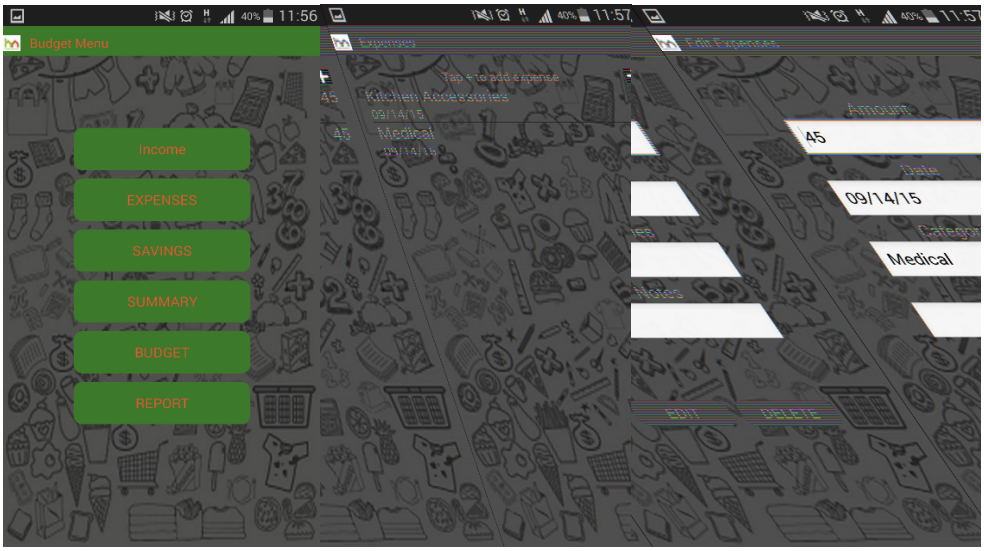


1.User selects income button, income activity will be open.

2.User select add icon from the income activity to add income.



1. User selects the added income in from list view to edit income.

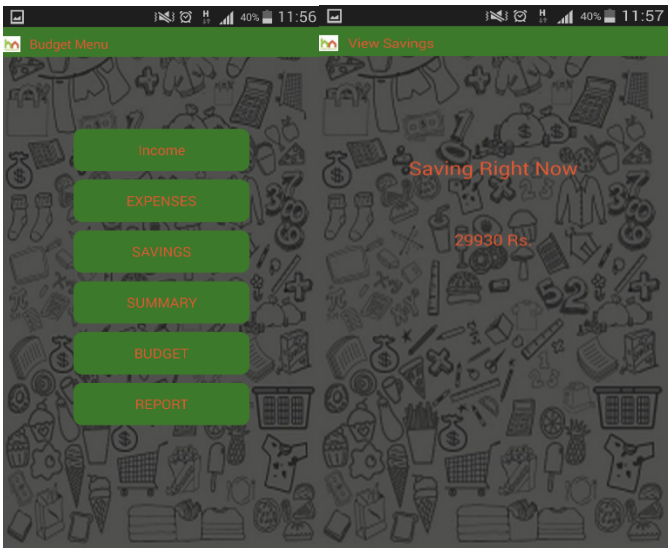


1.User selects expense button, expense activity will be open.

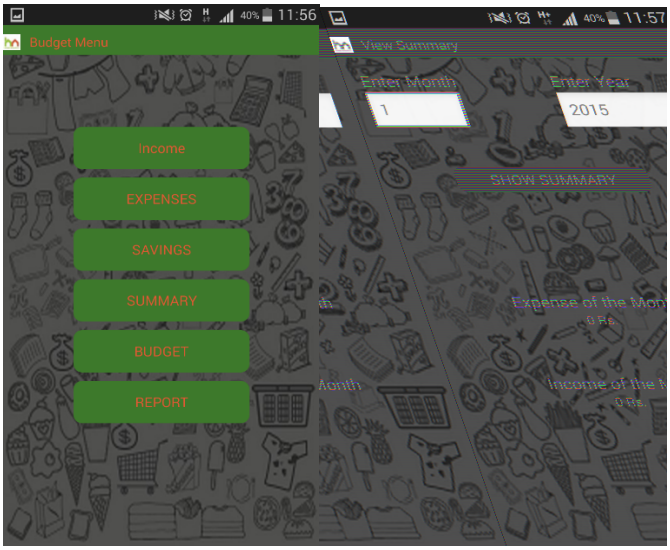
2.User select add icon from the expense activity to add expense.



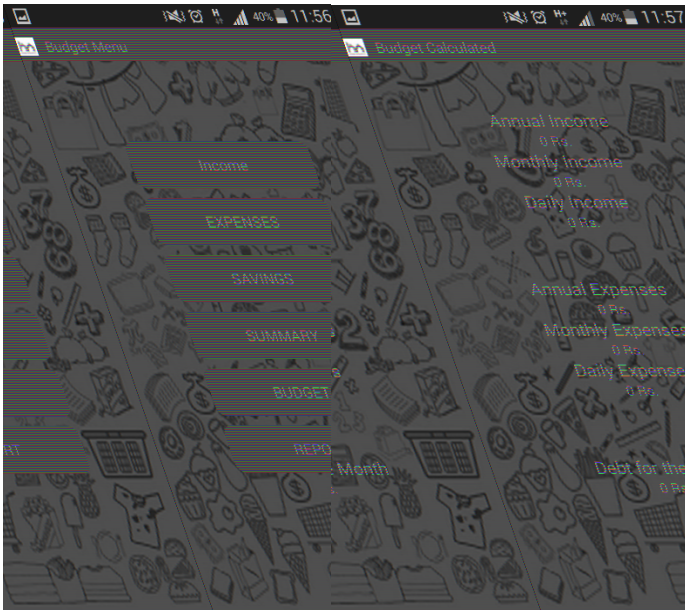
1. User selects the added expense in from list view to edit expense.



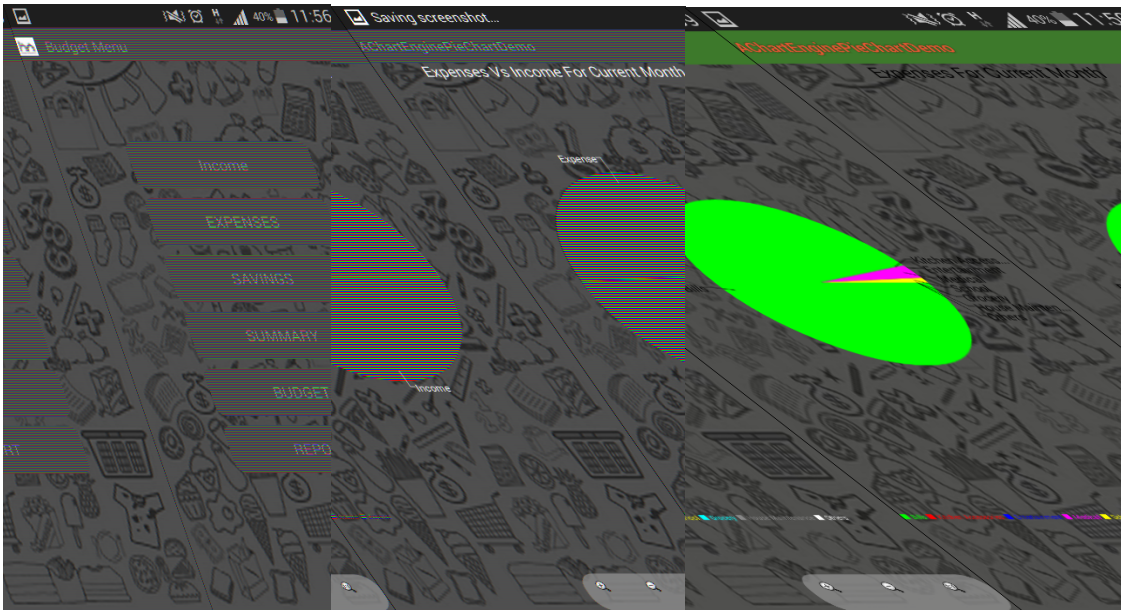
1. User select saving to view the amount that is left.



1. User select summary an activity will be opened.
2. User enter month anr year to view summary.

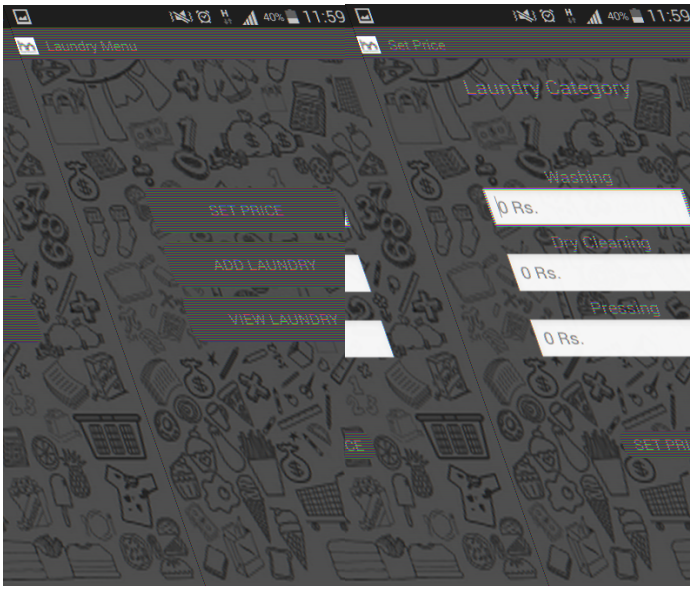


1. User select budget the calculated budget will b show in another activity.

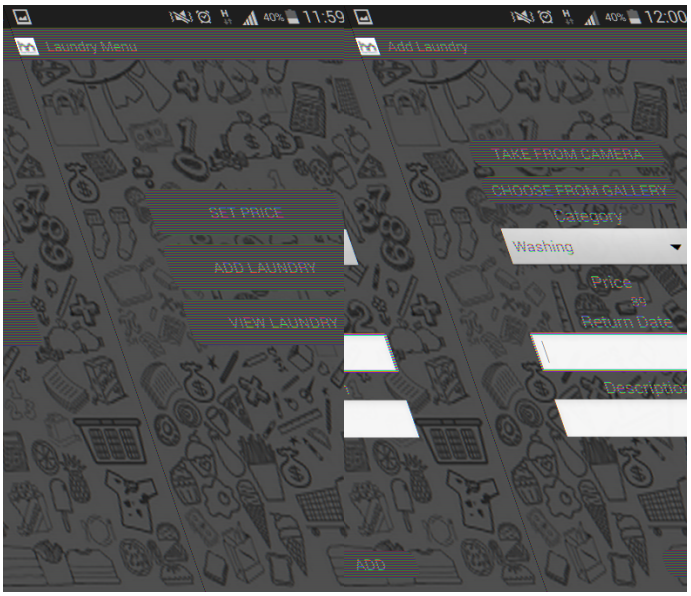


1. User select reports and will be able to view pie chart for amount added for income and expenses.

4.Laundry Management:

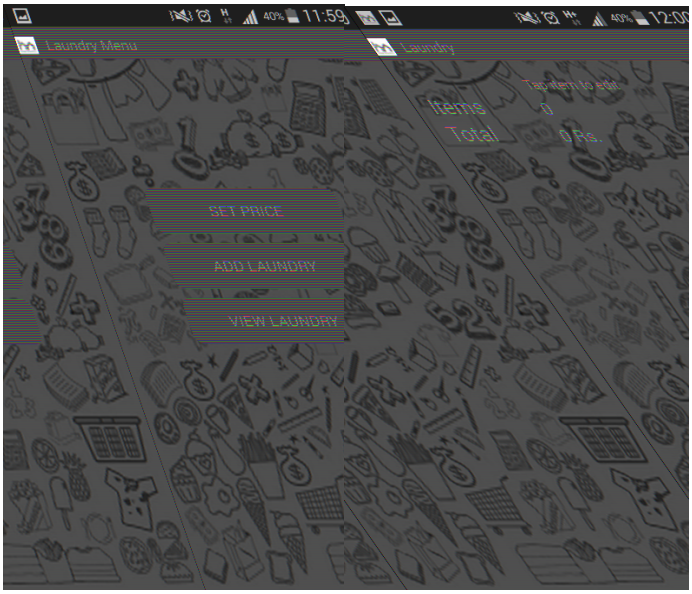


1. User select set price then will be able to enter price for laundry categories and set price.



1.User select add laundry then add laundry will be opened.

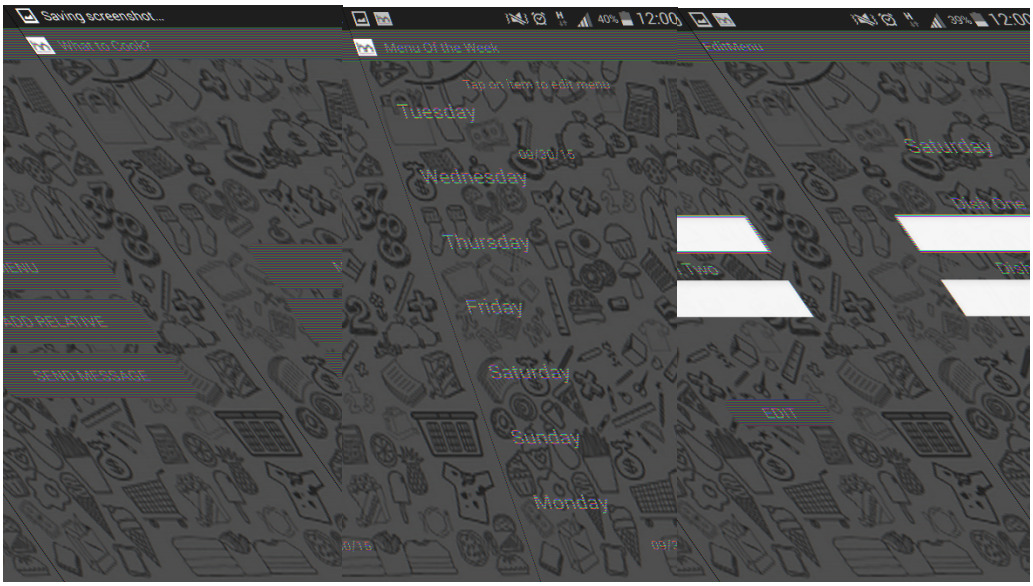
2.User take the picture of dress and add details then select add to save information.



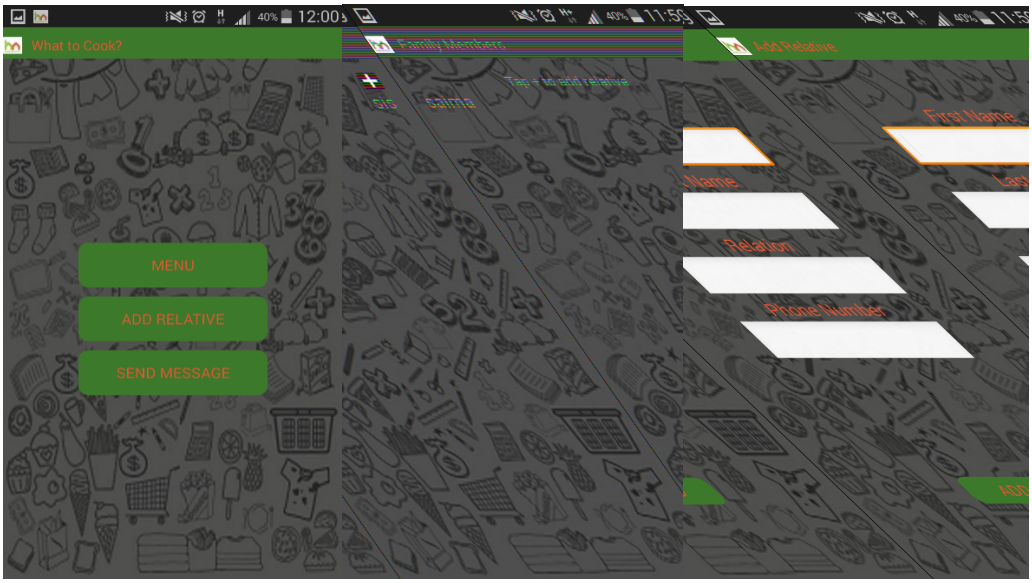
1.User select view laundry then laundry will be opened.

2.User can view number of dresses and their price total in listview.

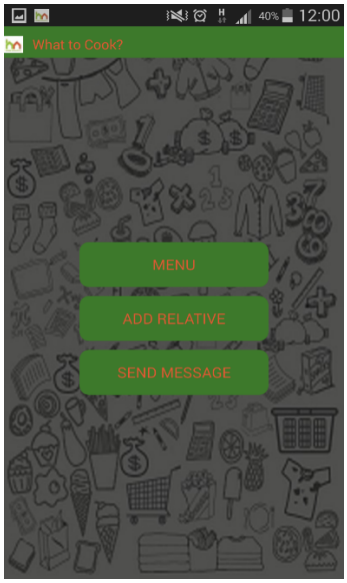
5. Dish of the Day:



1. User select menu then menu of week will be opened.
2. User tab the day of week for which dishes will be set and menu will be edited.



1. User can add relatives and their numbers.
2. User adds information save them.



1 User select send message to get the dishes decided from the relatives added.

APPENDIX B

REFRENCES

References

1. **Diagramming and Collaboration, August, 2015. URL:www.creately.com**
2. **Android development tutorial URL: http://developer.android.com/training/index.html**
3. **Visual paradigm (UML diagrams)**
4. **Gimp 2**
5. **Book**
6. **https://en.wikipedia.org/wiki/Android**